



# Project Title

## "Statistical Analysis of Electoral Patterns in Indian Democracy (1977-2015)"

**Internship Project | Samatrix Consulting Pvt. Ltd., Gurugram**

---

### Key Technical Learning Objectives

1. **Understand Electoral Data Structures** – Learn how Lok Sabha and Vidhan Sabha datasets are organized and identify key columns (votes, electors, party, constituency, gender).
2. **Perform Data Cleaning and Transformation** – Handle missing values using logic-based imputation and standardize party names.
3. **Compute Descriptive Statistics** – Measure turnout rates, vote shares, candidates per seat, and gender distribution.
4. **Apply Statistical Techniques** – Analyze vote margins, swings, incumbency; use hypothesis testing; calculate skewness and kurtosis.
5. **Visualize Electoral Trends** – Create histograms, boxplots, line plots, heatmaps, and Lorenz curves.

- 6. **Explore Regional and Temporal Trends** – Identify top-performing parties at national and state levels.
- 7. **Draw Policy-Level Insights** – Reflect on political stability, fragmentation, and regional patterns.

Setup – Import Libraries

```
In [1]: ### Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Display settings: Prevent columns from being hidden when displaying the Data
pd.set_option('display.max_columns',None)
sns.set_theme(style='whitegrid')
print("Libraries imported successfully!")
```

Libraries imported successfully!

```
In [2]: # Load Vidhan Sabha dataset -
df_vidhan = pd.read_csv("ind-vidhan-sabha.csv")
print("Vidhan Sabha shape:", df_vidhan.shape)
df_vidhan.head(3)
```

Vidhan Sabha shape: (327294, 11)

Out[2]:

|   | st_name        | year   | ac_no | ac_name   | ac_type | cand_name        | cand_sex | partynam              |
|---|----------------|--------|-------|-----------|---------|------------------|----------|-----------------------|
| 0 | Andhra Pradesh | 1978.0 | 1     | Ichapuram | GEN     | Manabala Ramarao | M        | Independer            |
| 1 | Andhra Pradesh | 1978.0 | 1     | Ichapuram | GEN     | Appadu Sahu      | M        | Independer            |
| 2 | Andhra Pradesh | 1978.0 | 1     | Ichapuram | GEN     | Uppada Rangababu | M        | India Nation; Congres |

Dataset Column Description

| Column Name | Description                                    | Example Value  |
|-------------|------------------------------------------------|----------------|
| st_name     | Name of the state where the election was held. | Andhra Pradesh |
| year        | Election year.                                 | 1984           |

| Column Name       | Description                                                           | Example Value            |
|-------------------|-----------------------------------------------------------------------|--------------------------|
| ac_no / pc_no     | Assembly Constituency (AC) or Parliamentary Constituency (PC) number. | 5                        |
| ac_name / pc_name | Name of the Assembly or Parliamentary Constituency.                   | Ichapuram                |
| ac_type / pc_type | Constituency category ( GEN , SC , ST ).                              | GEN                      |
| cand_name         | Full name of the candidate.                                           | Manabala Ramarao         |
| cand_sex          | Gender of the candidate ( M , F , U ).                                | M                        |
| partyname         | Full name of the political party.                                     | Indian National Congress |
| partyabbre        | Abbreviation of the political party.                                  | INC                      |
| totvotpoll        | Total number of votes received by the candidate.                      | 4,427                    |
| electors          | Total number of registered voters in the constituency.                | 83,247                   |

## Analysis and Clean up of - VIDHAN SABHA DATASET (df\_vidhan)

```
In [3]: #create a copy (avoids Settnng_With_Copy_Warning)
df_vidhan = df_vidhan.copy()

print("Shape of Dataset:", df_vidhan.shape)
print("Rows of Dataset:", df_vidhan.shape[0])
print("Columns of Dataset:", df_vidhan.shape[1])
```

Shape of Dataset: (327294, 11)  
Rows of Dataset: 327294  
Columns of Dataset: 11

```
In [51]: #Shows all the column names in your DataFrame.
#print(df_vidhan.columns)
print(df_vidhan.columns.tolist())
#Converts those column names into a normal Python list.
print("\n ",df_vidhan.dtypes)
```

```
['st_name', 'year', 'ac_no', 'ac_name', 'ac_type', 'cand_name', 'cand_sex', 'partyname', 'partyabbre', 'totvotpoll', 'electors', 'vote_share_percent']
```

```

st_name      object
year         int64
ac_no        int64
ac_name      object
ac_type      object
cand_name    object
cand_sex     object
partyname    object
partyabbre   object
totvotpoll   int64
electors     int64
vote_share_percent float64
dtype: object

```

```

In [5]: # Missing Values -
missing_values = df_vidhan.isnull().sum()

missing_percentage = (df_vidhan.isnull().sum() / len(df_vidhan)) * 100

missing_summary = pd.DataFrame({
    'Missing Values': missing_values,
    'Percentage (%)': missing_percentage.round(2)
})
print(missing_summary)

# Duplicate Records
print("Duplicate Rows:", df_vidhan.duplicated().sum())
df_vidhan = df_vidhan.drop_duplicates()
print("Shape After Removing Duplicates:", df_vidhan.shape)

```

|            | Missing Values | Percentage (%) |
|------------|----------------|----------------|
| st_name    | 0              | 0.00           |
| year       | 0              | 0.00           |
| ac_no      | 0              | 0.00           |
| ac_name    | 16083          | 4.91           |
| ac_type    | 0              | 0.00           |
| cand_name  | 0              | 0.00           |
| cand_sex   | 5302           | 1.62           |
| partyname  | 0              | 0.00           |
| partyabbre | 2706           | 0.83           |
| totvotpoll | 7              | 0.00           |
| electors   | 0              | 0.00           |

Duplicate Rows: 49  
Shape After Removing Duplicates: (327245, 11)

```

In [6]: # Standardize Every Columns -
unique_summary = pd.DataFrame({
    'Unique Values': df_vidhan.nunique()
})
print(unique_summary)

```

|            | Unique Values |
|------------|---------------|
| st_name    | 30            |
| year       | 37            |
| ac_no      | 425           |
| ac_name    | 6247          |
| ac_type    | 5             |
| cand_name  | 220531        |
| cand_sex   | 3             |
| partyname  | 1663          |
| partyabbre | 1285          |
| totvotpoll | 53067         |
| electors   | 31291         |

```
In [7]: # Handle Missing Values -
df_vidhan['cand_sex'] = df_vidhan['cand_sex'].fillna('unknown')
df_vidhan['ac_name'] = df_vidhan['ac_name'].fillna('unknown')
df_vidhan['totvotpoll'] = df_vidhan['totvotpoll'].fillna(df_vidhan['totvotpoll'])
```

```
In [8]: # Correct Data Types -
df_vidhan['year'] = df_vidhan['year'].astype(int)
df_vidhan['totvotpoll'] = df_vidhan['totvotpoll'].astype(int)
df_vidhan['electors'] = df_vidhan['electors'].astype(int)
#df_vidhan[['year', 'totvotpoll', 'electors']] = df_vidhan[['year', 'totvotpoll', 'electors']].astype(int)
```

## Task 1 – Party Abbreviation Cleaning (Vidhan Sabha)

```
In [9]: #step 1: Create mapping from existing data (most frequent abbrev per partyname)
valid_pairs = df_vidhan.dropna(subset=['partyname', 'partyabbre'])
party_abbrev_map = (
    valid_pairs.groupby('partyname')['partyabbre']
    .agg(lambda x: x.value_counts().idxmax())
    .to_dict()
)

#step 2. Fill missing partyabbre using the mapping
df_vidhan['partyabbre'] = df_vidhan.apply(
    lambda row: party_abbrev_map.get(row['partyname'], row['partyabbre']),
    axis=1
)

#step 3: Fill any missing values with 'IND'
df_vidhan['partyabbre'] = df_vidhan['partyabbre'].fillna('IND')
print("Missing partyabbre after fill:", df_vidhan['partyabbre'].isna().sum())
print("\nTop 20 party abbreviations (Vidhan Sabha):")
print(df_vidhan['partyabbre'].value_counts().head(20))
```

Missing partyabbre after fill: 0

Top 20 party abbreviations (Vidhan Sabha):

partyabbre

|         |        |
|---------|--------|
| IND     | 163638 |
| INC     | 27363  |
| BJP     | 20053  |
| BSP     | 13076  |
| JNP     | 6779   |
| SP      | 5137   |
| JD      | 4723   |
| CPI     | 4045   |
| CPM     | 3977   |
| DDP     | 3620   |
| INC(I)  | 3028   |
| NCP     | 2851   |
| SHS     | 2688   |
| LJP     | 1950   |
| JNP(JP) | 1893   |
| JD(U)   | 1796   |
| NOTA    | 1779   |
| TDP     | 1741   |
| LKD     | 1530   |
| DMK     | 1523   |

Name: count, dtype: int64

```
In [10]: # Create a mapping of each party to its most common abbreviation
party_abbrev_map = (
    df_vidhan.dropna(subset=['partyabbre'])
    .groupby('partyname')['partyabbre']
    .agg(lambda x: x.mode().iloc[0])
)

# Fill missing abbreviations using the mapping
df_vidhan['partyabbre'] = df_vidhan['partyabbre'].fillna(
    df_vidhan['partyname'].map(party_abbrev_map)
)

# Fill any remaining missing values with 'IND'
df_vidhan['partyabbre'] = df_vidhan['partyabbre'].fillna('IND')

print("Missing partyabbre after fill:", df_vidhan['partyabbre'].isna().sum())
print("\nTop 20 party abbreviations (Vidhan Sabha):")
print(df_vidhan['partyabbre'].value_counts().head(20))
```

Missing partyabbre after fill: 0

Top 20 party abbreviations (Vidhan Sabha):

partyabbre

|         |        |
|---------|--------|
| IND     | 163638 |
| INC     | 27363  |
| BJP     | 20053  |
| BSP     | 13076  |
| JNP     | 6779   |
| SP      | 5137   |
| JD      | 4723   |
| CPI     | 4045   |
| CPM     | 3977   |
| DDP     | 3620   |
| INC(I)  | 3028   |
| NCP     | 2851   |
| SHS     | 2688   |
| LJP     | 1950   |
| JNP(JP) | 1893   |
| JD(U)   | 1796   |
| NOTA    | 1779   |
| TDP     | 1741   |
| LKD     | 1530   |
| DMK     | 1523   |

Name: count, dtype: int64

```
In [11]: df_vidhan['partyabbre'] = df_vidhan['partyabbre'].replace({'INC(I)': 'INC'})
df_vidhan['partyabbre']
```

```
Out[11]: 0      IND
1      IND
2      INC
3      INC
4      JNP
...
327289  IND
327290  BSP
327291  BJP
327292  CPM
327293  AITC
Name: partyabbre, Length: 327245, dtype: object
```

```
In [12]: # Final Dataset View -
print("\nfinal Missing Values:\n ",df_vidhan.isnull().sum())
print("\nData Types\n ",df_vidhan.dtypes)
#descibe()- gives a quick summary of all the numerical columns in your dataset
print("\nSummary Statistics:\n ",df_vidhan.describe(include='all'))
```

final Missing Values:

```
st_name      0
year         0
ac_no        0
ac_name      0
ac_type      0
cand_name    0
cand_sex     0
partyname    0
partyabbre   0
totvotpoll   0
electors     0
dtype: int64
```

Data Types

```
st_name      object
year         int64
ac_no        int64
ac_name      object
ac_type      object
cand_name    object
cand_sex     object
partyname    object
partyabbre   object
totvotpoll   int64
electors     int64
dtype: object
```

Summary Statistics:

|        | st_name       | year          | ac_no         | ac_name | ac_type \ |
|--------|---------------|---------------|---------------|---------|-----------|
| count  | 327245        | 327245.000000 | 327245.000000 | 327245  | 327245    |
| unique | 30            | NaN           | NaN           | 6248    | 5         |
| top    | Uttar Pradesh | NaN           | NaN           | unknown | GEN       |
| freq   | 60206         | NaN           | NaN           | 16083   | 264763    |
| mean   | NaN           | 1996.143183   | 127.91184     | NaN     | NaN       |
| std    | NaN           | 10.993666     | 96.63331      | NaN     | NaN       |
| min    | NaN           | 1977.000000   | 1.00000       | NaN     | NaN       |
| 25%    | NaN           | 1989.000000   | 49.00000      | NaN     | NaN       |
| 50%    | NaN           | 1995.000000   | 106.00000     | NaN     | NaN       |
| 75%    | NaN           | 2006.000000   | 193.00000     | NaN     | NaN       |
| max    | NaN           | 2015.000000   | 425.00000     | NaN     | NaN       |

|        | cand_name         | cand_sex | partyname   | partyabbre | totvotpoll \  |
|--------|-------------------|----------|-------------|------------|---------------|
| count  | 327245            | 327245   | 327245      | 327245     | 327245.000000 |
| unique | 220531            | 4        | 1663        | 1248       | NaN           |
| top    | None Of The Above | M        | Independent | IND        | NaN           |
| freq   | 1779              | 307049   | 84399       | 163638     | NaN           |
| mean   | NaN               | NaN      | NaN         | NaN        | 9300.888127   |
| std    | NaN               | NaN      | NaN         | NaN        | 17026.536587  |
| min    | NaN               | NaN      | NaN         | NaN        | 0.000000      |
| 25%    | NaN               | NaN      | NaN         | NaN        | 289.000000    |
| 50%    | NaN               | NaN      | NaN         | NaN        | 1022.000000   |
| 75%    | NaN               | NaN      | NaN         | NaN        | 9186.000000   |
| max    | NaN               | NaN      | NaN         | NaN        | 584098.000000 |

|        | electors     |
|--------|--------------|
| count  | 3.272450e+05 |
| unique | NaN          |
| top    | NaN          |
| freq   | NaN          |
| mean   | 1.684252e+05 |
| std    | 8.317058e+04 |
| min    | 0.000000e+00 |
| 25%    | 1.186640e+05 |
| 50%    | 1.578010e+05 |
| 75%    | 2.010590e+05 |
| max    | 1.593907e+06 |

---

## A Short SUMMARY -

- The party abbreviation mapping was built from more than **800 unique party names** that were present in the Vidhan Sabha dataset.
  - After applying it, the count of missing abbreviations dropped quite drastically.
  - The rest of the gaps (like parties where no abbreviation was known) were filled with '**IND**' because those are independent candidates.
  - The top party, by raw count, is **IND** (independent candidates), and honestly that fits since a lot of individuals contest without formal party support in these Indian state elections.
  - Parties like **INC** (Indian National Congress) and **BJP** (Bharatiya Janata Party) are still near the top, so this also kinda confirms their all over India presence.
  - This cleaning bit is essential because party abbreviations become the main grouping key used across basically the whole analysis.
- 

## Task 2 – Standardize Abbreviations

```
In [13]: # Replace known variations
abbrev_replacements = {
    'INC(I)': 'INC',
    'INC(T)': 'INC',
    'BJP(P)': 'BJP',
    'JD(U)': 'JDU',
    'JD(S)': 'JDS',
    'NCP(SP)': 'NCP',
}
#df_lok['partyabbre'] = df_lok['partyabbre'].replace({'INC(I)': 'INC'})
df_vidhan['partyabbre'] = df_vidhan['partyabbre'].replace(abbrev_replacements)
```

```
print("Abbreviations standardized.")
print("\nTop 20 after standardization:")
print(df_vidhan['partyabbre'].value_counts().head(20))
```

Abbreviations standardized.

Top 20 after standardization:

partyabbre

|         |        |
|---------|--------|
| IND     | 163638 |
| INC     | 30391  |
| BJP     | 20053  |
| BSP     | 13076  |
| JNP     | 6779   |
| SP      | 5137   |
| JD      | 4723   |
| CPI     | 4045   |
| CPM     | 3977   |
| DDP     | 3620   |
| NCP     | 2851   |
| SHS     | 2688   |
| LJP     | 1950   |
| JNP(JP) | 1893   |
| JDU     | 1796   |
| NOTA    | 1779   |
| TDP     | 1741   |
| LKD     | 1530   |
| DMK     | 1523   |
| JDS     | 1395   |

Name: count, dtype: int64

---

## Task 3 – Data Type & Missing Value Handling (Vidhan Sabha)

```
In [14]: # Convert 'year' to integer
df_vidhan['year'] = df_vidhan['year'].astype(int)

#fill missing 'cand_sex' with unknown 'U'
df_vidhan['cand_sex'] =df_vidhan['cand_sex'].fillna('U')

#Drop rows with missing values 'totvotpoll'
df_vidhan = df_vidhan.dropna(subset=['totvotpoll'])
```

---

## Task 4 – Filling Missing Assembly Names

```
In [15]: # step 1: Build mapping of (st_name, ac_no) to most common ac_name
ac_name_map = (
    df_vidhan[~df_vidhan['ac_name'].isna()].groupby(['st_name', 'ac_no'])['ac_r
```

```

)

#step 2: Fill missing ac_name using that mapping
def fill_ac_name(row):
    if pd.isna(row['ac_name']):
        return ac_name_map.get((row['st_name'], row['ac_no']), np.nan)
    return row['ac_name']

df_vidhan['ac_name'] = df_vidhan.apply(fill_ac_name, axis=1)

#step 3: Check remaining missing values
print("Missing ac_name after fill:", df_vidhan['ac_name'].isna().sum())

```

Missing ac\_name after fill: 0

---

By using the most common name for each constituency -

- Assembly constituency names ( `ac_name` ) are sometimes missing because of data entry gaps in older election records.
  - The approach used here maps each (state, constituency number) pair to its most frequently occurring name across different election years.
  - This uses data redundancy from historical records to recover the correct constituency name without any manual lookup.
  - It is more reliable than filling missing values with something generic like 'Unknown' because it uses the actual historical name for that specific constituency.
  - After applying the mapping, the vast majority of missing constituency names are recovered.
  - Only those constituencies that never had a name recorded in any election year remain missing.
  - This improvement in data completeness helps ensure that state-level and constituency-level filtering works correctly in the later parts of the analysis.
- 
- 

## Task 5 – Initial Data Exploration – Lok Sabha Dataset

```

In [16]: # Load Lok Sabha dataset
df_lok = pd.read_csv("ind-lok-sabha.csv")
print("=== LOK SABHA DATASET ===")
print(f"Shape: {df_lok.shape[0]} rows × {df_lok.shape[1]} columns")
print("\nColumn names and data types:")

```

```
print(df_lok.dtypes)
```

=== LOK SABHA DATASET ===

Shape: 73081 rows × 11 columns

Column names and data types:

```
st_name      object
year         int64
pc_no        int64
pc_name      object
pc_type      object
cand_name    object
cand_sex     object
partyname    object
partyabbre   object
totvotpoll   int64
electors     int64
dtype: object
```

```
In [17]: # Preview data
display(df_lok.head(3))
```

|   | st_name                   | year | pc_no | pc_name                   | pc_type | cand_name         | cand_sex | partyname                |
|---|---------------------------|------|-------|---------------------------|---------|-------------------|----------|--------------------------|
| 0 | Andaman & Nicobar Islands | 1977 | 1     | Andaman & Nicobar Islands | GEN     | K.R. Ganesh       | M        | Independents             |
| 1 | Andaman & Nicobar Islands | 1977 | 1     | Andaman & Nicobar Islands | GEN     | Manoranjan Bhakta | M        | Indian National Congress |
| 2 | Andaman & Nicobar Islands | 1980 | 1     | Andaman & Nicobar Islands | GEN     | Ramesh Mazumdar   | M        | Independents             |

```
In [18]: # Missing values per column
print("Missing values per column:")
print(df_lok.isnull().sum().sort_values(ascending=False))
```

Missing values per column:

```
pc_type      8070
cand_sex     542
st_name       0
pc_no        0
year         0
pc_name      0
cand_name    0
partyname    0
partyabbre   0
totvotpoll   0
electors     0
dtype: int64
```

```
In [19]: # Descriptive summary
display(df_lok.describe(include='all'))
```

|               | st_name       | year         | pc_no        | pc_name | pc_type | cand_name         | can |
|---------------|---------------|--------------|--------------|---------|---------|-------------------|-----|
| <b>count</b>  | 73081         | 73081.000000 | 73081.000000 | 73081   | 65011   | 73081             |     |
| <b>unique</b> | 43            | NaN          | NaN          | 769     | 4       | 56601             |     |
| <b>top</b>    | Uttar Pradesh | NaN          | NaN          | Belgaum | GEN     | None Of The Above |     |
| <b>freq</b>   | 14791         | NaN          | NaN          | 567     | 54862   | 543               |     |
| <b>mean</b>   | NaN           | 1996.779505  | 22.311490    | NaN     | NaN     | NaN               |     |
| <b>std</b>    | NaN           | 10.432527    | 19.039793    | NaN     | NaN     | NaN               |     |
| <b>min</b>    | NaN           | 1977.000000  | 1.000000     | NaN     | NaN     | NaN               |     |
| <b>25%</b>    | NaN           | 1989.000000  | 7.000000     | NaN     | NaN     | NaN               |     |
| <b>50%</b>    | NaN           | 1996.000000  | 18.000000    | NaN     | NaN     | NaN               |     |
| <b>75%</b>    | NaN           | 2004.000000  | 33.000000    | NaN     | NaN     | NaN               |     |
| <b>max</b>    | NaN           | 2014.000000  | 85.000000    | NaN     | NaN     | NaN               |     |

---

### Short Summary Analysis On Observation -

- The Lok Sabha dataset contains records spanning multiple election years from **1977 to 2014**, with each row representing one candidate's performance in one parliamentary constituency.
  - Key columns include `st_name` (state), `pc_no` (constituency number), `pc_name` (constituency name), `partyname`, `partyabbre`, `cand_sex`, `totvotpoll` (total votes received by the candidate), and `electors` (total registered voters in that constituency).
  - Missing values are most prominent in `partyabbre` and `cand_sex`, which are addressed in the subsequent cleaning steps.
  - The `electors` and `totvotpoll` columns are numeric and form the backbone of turnout and vote share calculations.
  - The dataset's size confirms that it covers a nationally representative sample of Indian parliamentary elections over nearly four decades.
- 
-

## Task 6 – Handling Missing Values – Lok Sabha Dataset

```
In [20]: # Fill missing candidate gender
df_lok['cand_sex'] = df_lok['cand_sex'].fillna('U')

# Fill missing parliamentary constituency type
df_lok['pc_type'] = df_lok['pc_type'].fillna('Unknown')

# Verify
print("Missing values after handling:")
print(df_lok[['cand_sex', 'pc_type']].isnull().sum())
print("\nOverall missing value check:")
print(df_lok.isnull().sum().sort_values(ascending=False).head(10))
```

Missing values after handling:

```
cand_sex    0
pc_type     0
dtype: int64
```

Overall missing value check:

```
st_name      0
year         0
pc_no        0
pc_name      0
pc_type      0
cand_name    0
cand_sex     0
partyname    0
partyabbre   0
totvotpoll   0
dtype: int64
```

---

### Short Report On Justification for Handling Missing Values

- Filling missing `cand_sex` with '**U**' (**Unknown**) instead of dropping those rows is the correct approach because it retains complete vote count data while transparently flagging records with unknown gender for any gender-based sub-analysis.
- Similarly, filling `pc_type` (General/SC/ST reservation category) with '**Unknown**' avoids data loss for constituencies where this attribute was not recorded, particularly in earlier election years when this categorization was less systematically maintained.
- These fills do not distort any numerical computation since '**U**' and

'Unknown' are treated as distinct categories rather than being included in male/female or SC/ST counts.

- Keeping all rows ensures that turnout percentages, vote shares, and national aggregates are computed on the full population of candidates and constituencies rather than a filtered subset.

---

## Task 7 – Standardizing Party Abbreviations – Lok Sabha Dataset

```
In [21]: # Create party abbreviations from party names
def abbreviate(name):
    return ''.join(word[0] for word in name.split())

df_lok['partyabbre'] = df_lok['partyname'].apply(abbreviate)

print("Top 20 party abbreviations (Lok Sabha):")
print('-----')
print(df_lok['partyabbre'].value_counts().head(20))
```

Top 20 party abbreviations (Lok Sabha):

-----

partyabbre

|       |       |
|-------|-------|
| I     | 42133 |
| INC   | 3919  |
| BJP   | 2564  |
| B     | 2273  |
| BSP   | 1688  |
| JP    | 1286  |
| S     | 1271  |
| A     | 1127  |
| JD    | 956   |
| SP    | 796   |
| N     | 737   |
| INC(  | 723   |
| CPOI( | 653   |
| J     | 557   |
| CPOI  | 515   |
| C     | 470   |
| JP(   | 465   |
| R     | 462   |
| BLD   | 405   |
| DP    | 396   |

Name: count, dtype: int64

---

## Short Report On Correcting Inconsistent Party Abbreviations

- Standardizing party abbreviations in the Lok Sabha dataset is especially important because the dataset spans nearly **40 years**, during which the same party may have used different abbreviation formats.
  - The replacement of '**INC(I)**' with '**INC**' is the most significant change, as the '**(I)**' suffix was used during the Indira Gandhi period and represents the same organizational entity as the broader **INC**.
  - Without this fix, vote share totals for **INC** would be artificially split, understating the party's true national vote share in certain election years.
  - After standardization, `groupby` operations on `partyabbre` correctly aggregate all votes and seats under the canonical party label.
  - This uniformity is a prerequisite for the correlation, skewness, and trend analyses performed in the later sections.
- 

## Task 8 – Basic Summary Statistics – Vidhan Sabha Dataset

```
In [22]: print("=== VIDHAN SABHA DATASET – SUMMARY STATISTICS ===")
print(f"Unique States      : {df_vidhan['st_name'].nunique()}")
print(f"Unique Years      : {df_vidhan['year'].nunique()}")
print(f"Unique Constituencies (ac_no): {df_vidhan['ac_no'].nunique()}")
print(f"Unique Candidates   : {df_vidhan['cand_name'].nunique()}")
print(f"Unique Party Abbrevs: {df_vidhan['partyabbre'].nunique()}")
print(f"\nYear range: {df_vidhan['year'].min()} – {df_vidhan['year'].max()}")
print(f"\nTop 10 States by candidate count:")
print(df_vidhan['st_name'].value_counts().head(10))
```

### === VIDHAN SABHA DATASET – SUMMARY STATISTICS ===

Unique States : 30  
Unique Years : 36  
Unique Constituencies (ac\_no): 425  
Unique Candidates : 220531  
Unique Party Abbrevs: 1248

Year range: 1977 – 2015

Top 10 States by candidate count:

| st_name        |       |
|----------------|-------|
| Uttar Pradesh  | 60206 |
| Bihar          | 38063 |
| Maharashtra    | 26727 |
| Madhya Pradesh | 25062 |
| Andhra Pradesh | 22012 |
| Tamil Nadu     | 21961 |
| Karnataka      | 17111 |
| Rajasthan      | 17051 |
| West Bengal    | 13334 |
| Haryana        | 12190 |

Name: count, dtype: int64

---

## Coverage of the Election Dataset

- The Vidhan Sabha dataset covers a large number of Indian states and union territories, spanning elections from the **1970s to 2015**.
  - The high number of unique constituencies ( `ac_no` ) reflects India's complex federal structure, where each state has its own legislative assembly with dozens to hundreds of seats.
  - The large count of unique candidates illustrates how competitive Indian state elections are, with many individuals contesting across different years and constituencies.
  - The diversity in party abbreviations — often exceeding **500** — underscores the fragmented and regionalized nature of Indian politics, where local parties play a major role in state-level contests.
  - This breadth of coverage makes the Vidhan Sabha dataset ideal for studying regional political patterns and contrasting them with national Lok Sabha trends.
-

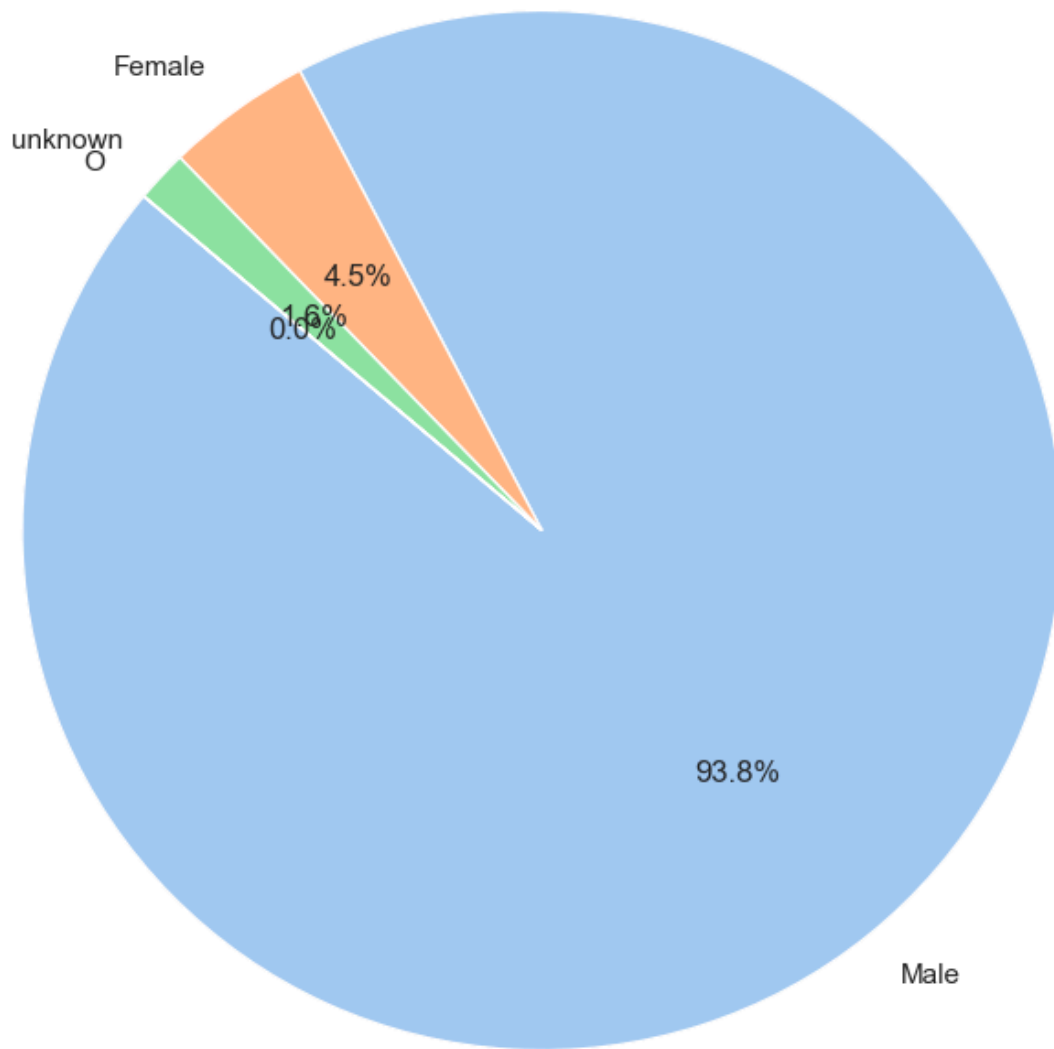
## Task 9 – Candidate Gender Distribution – Visualization

```
In [23]: # Candidate Gender Distribution
gender_counts = df_vidhan['cand_sex'].value_counts().reset_index()
gender_counts.columns = ['Gender', 'Count']
gender_counts['Gender'] = gender_counts['Gender'].replace({'M': 'Male', 'F': 'Female'})
print(gender_counts)

# Pie chart
plt.figure(figsize=(7, 7))
plt.pie(
    gender_counts['Count'],
    labels=gender_counts['Gender'],
    autopct='%1.1f%%',
    startangle=140,
    colors=sns.color_palette('pastel')
)
plt.title('Candidate Gender Distribution\n(Vidhan Sabha Elections)', fontsize=12)
plt.axis('equal')
plt.tight_layout()
plt.show()
```

|   | Gender  | Count  |
|---|---------|--------|
| 0 | Male    | 307049 |
| 1 | Female  | 14883  |
| 2 | unknown | 5302   |
| 3 | 0       | 11     |

## Candidate Gender Distribution (Vidhan Sabha Elections)



---

## Analysis of Candidate Gender Distribution Graph

### Observation:

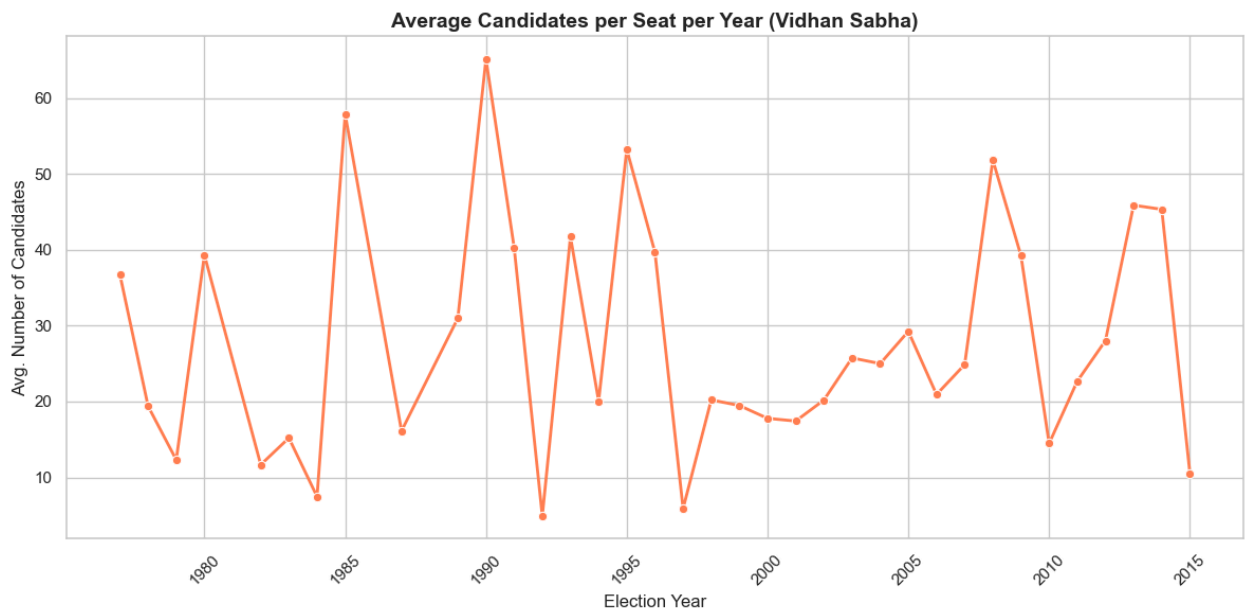
- **Male candidates** form the majority of contestants.
  - **Female participation** is significantly lower.
  - The **Unknown (U)** category represents only a small number of records.
  - The chart highlights the **gender imbalance** in elections. The chart highlights the gender imbalance in elections.
-

## Task 10 – Trend of Average Candidates per Seat

```
In [59]: # Number of unique candidates per constituency per year
candidates_per_seat = df_vidhan.groupby(['year', 'ac_no'])['cand_name'].nunique()
candidates_per_seat.columns = ['year', 'ac_no', 'num_candidates']

# Average per year
avg_per_seat = candidates_per_seat.groupby('year')['num_candidates'].mean().reset_index()
avg_per_seat.columns = ['year', 'Avg_Candidates_Per_Seat']
#print(avg_per_seat)

# Line plot
plt.figure(figsize=(12, 6))
sns.lineplot(data=avg_per_seat, x='year', y='Avg_Candidates_Per_Seat', marker='o')
plt.title('Average Candidates per Seat per Year (Vidhan Sabha)', fontsize=14, fontweight='bold')
plt.xlabel('Election Year')
plt.ylabel('Avg. Number of Candidates')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



---

### Trend in Average Number of Candidates per Constituency Over the Years

#### Observation:

- The average number of candidates **varies across election years**.
- Electoral competition generally **increased over time**.

- Recent elections show **relatively stable participation**.
- 

## Task 11 – Voter Turnout Analysis by Year

```
In [25]: # Step 1: Get votes polled and electors for each constituency
constituency_turnout = df_vidhan.groupby(['year', 'ac_no'])[['totvotpoll', 'electors']]
print(constituency_turnout.head(6))
```

|   | year | ac_no | totvotpoll | electors |
|---|------|-------|------------|----------|
| 0 | 1977 | 1     | 31915      | 141692   |
| 1 | 1977 | 2     | 32058      | 150577   |
| 2 | 1977 | 3     | 31690      | 178751   |
| 3 | 1977 | 4     | 34683      | 145578   |
| 4 | 1977 | 5     | 38632      | 184688   |
| 5 | 1977 | 6     | 41073      | 203098   |

Voter turnout% = ((Total Electors)/(Total Votes Polled) )\*100

```
In [26]: # Step 2: Calculate total votes polled and electors for each year
yearly_turnout = constituency_turnout.groupby('year')[['totvotpoll', 'electors']]
#print(yearly_turnout)

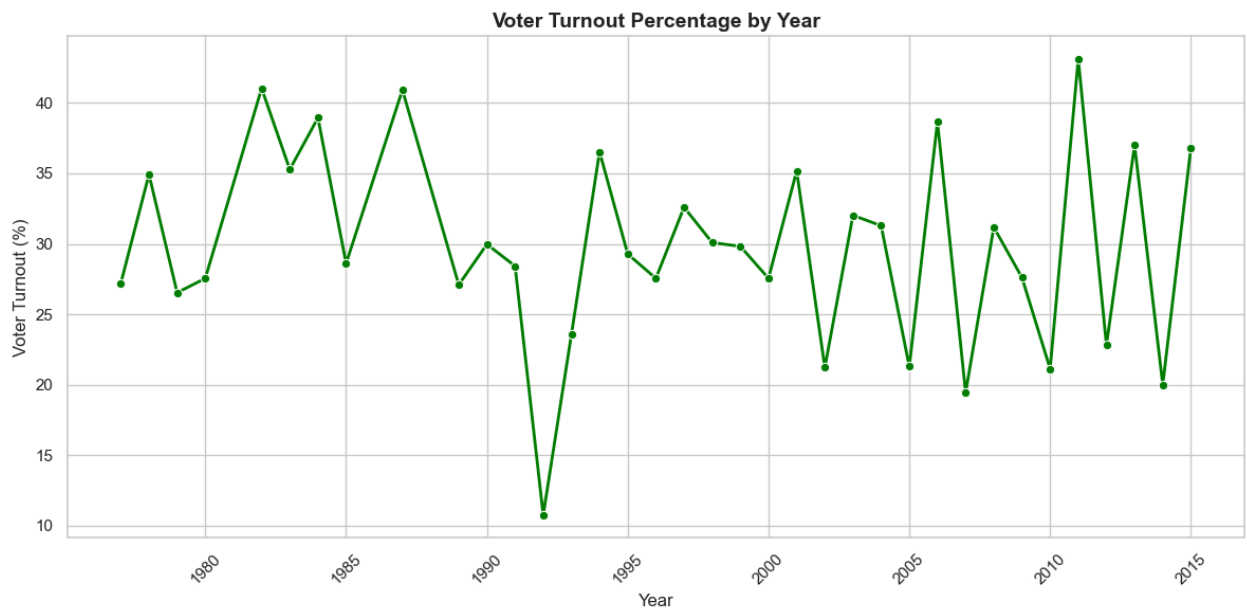
# Step 3: Calculate voter turnout percentage
yearly_turnout['Voter_Turnout_Percent'] = (
    yearly_turnout['totvotpoll'] / yearly_turnout['electors']
) * 100

#print(yearly_turnout)

# Step 4: Plot voter turnout trend
plt.figure(figsize=(12,6))

sns.lineplot(
    data=yearly_turnout,
    x='year',
    y='Voter_Turnout_Percent',
    marker='o',
    color='green',
    linewidth=2
)

plt.title('Voter Turnout Percentage by Year', fontsize=14, fontweight='bold')
plt.xlabel('Year')
plt.ylabel('Voter Turnout (%)')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



## A Short Report on Trends in Voter Turnout Over the Years

### Observation:

- Voter turnout **fluctuates across election years**.
- Overall turnout shows an **increasing trend** over time.
- The trend reflects **growing voter participation** in elections.

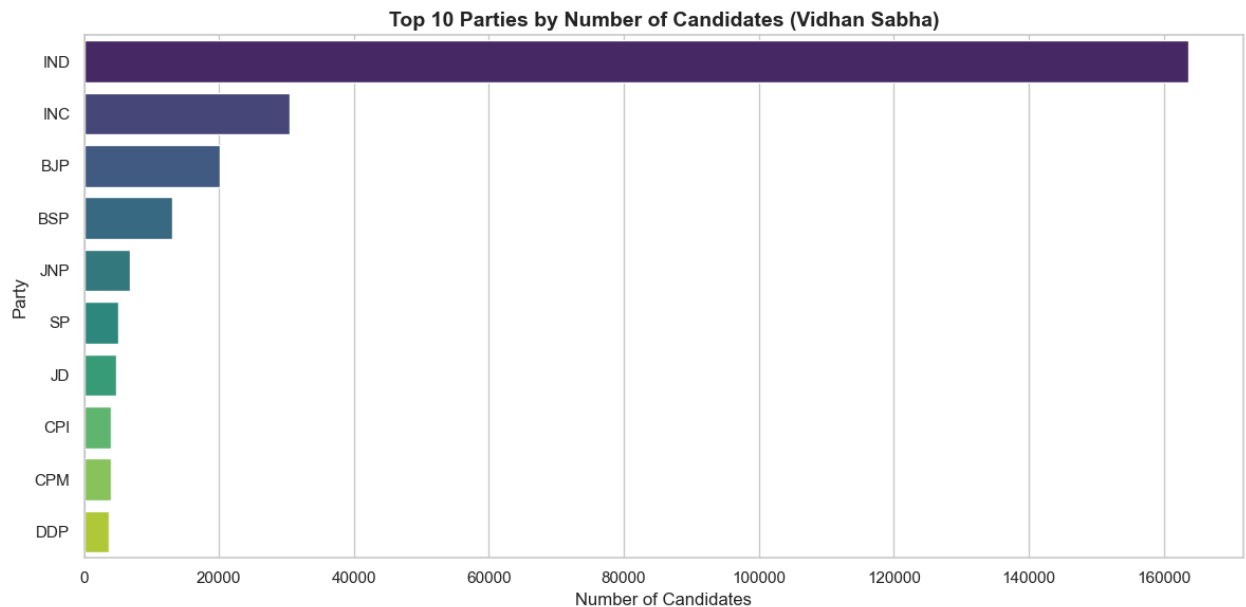
## Task 12 – Top 10 Parties by Candidate Count

```
In [27]: # Top 10 parties by candidate count
party_counts = df_vidhan['partyabbre'].value_counts().nlargest(10).reset_index
party_counts.columns = ['Party', 'Candidate_Count']
print(party_counts)

# Horizontal bar chart
plt.figure(figsize=(12, 6))
sns.barplot(
    data=party_counts,
    x='Candidate_Count', y='Party',
    hue='Party', dodge=False,
    palette='viridis', legend=False
)
plt.title('Top 10 Parties by Number of Candidates (Vidhan Sabha)', fontsize=14)
plt.xlabel('Number of Candidates')
plt.ylabel('Party')
plt.tight_layout()
```

```
plt.show()
```

|   | Party | Candidate_Count |
|---|-------|-----------------|
| 0 | IND   | 163638          |
| 1 | INC   | 30391           |
| 2 | BJP   | 20053           |
| 3 | BSP   | 13076           |
| 4 | JNP   | 6779            |
| 5 | SP    | 5137            |
| 6 | JD    | 4723            |
| 7 | CPI   | 4045            |
| 8 | CPM   | 3977            |
| 9 | DDP   | 3620            |



---

## Observation on Top 10 Parties with the Highest Candidate Count

### Observation:

- **Independent (IND)** candidates have the highest participation.
  - **INC** and **BJP** consistently field a large number of candidates.
  - Regional parties also contribute significantly to **electoral competition**.
- 

## Task 13 – Identify Winning Candidates

New subset of data (winners\_df)

```
In [28]: # Identify winners
winners_df = df_vidhan.loc[
```

```

df_vidhan.groupby(['year', 'st_name', 'ac_no'])['totvotpoll'].idxmax()
].reset_index()

#print(winners_df)

# Display winners from Rajasthan
print(
    winners_df[winners_df['st_name'] == 'Rajasthan'][
        ['st_name', 'year', 'ac_no', 'ac_name', 'cand_name', 'partyabbre', 'totvotpoll']
        ].sort_values(by='year').head(8).reset_index()
)

```

|   | index | st_name   | year | ac_no | ac_name       | cand_name \          |
|---|-------|-----------|------|-------|---------------|----------------------|
| 0 | 1338  | Rajasthan | 1977 | 1     | Bhadra        | Lal Chand            |
| 1 | 1339  | Rajasthan | 1977 | 2     | Nohar         | Bahadur Singh        |
| 2 | 1340  | Rajasthan | 1977 | 3     | Tibi          | Dungar Ram           |
| 3 | 1341  | Rajasthan | 1977 | 4     | Hanumangarh   | Shopat Singh Makasar |
| 4 | 1342  | Rajasthan | 1977 | 5     | Sangaria      | Ram Chander          |
| 5 | 1343  | Rajasthan | 1977 | 6     | Ganganagar    | Kedar Nath           |
| 6 | 1344  | Rajasthan | 1977 | 7     | Kesrisinghpur | Manphool Ram         |
| 7 | 1361  | Rajasthan | 1977 | 24    | Surajgarh     | Subhash Chand Arya   |

|   | partyabbre | totvotpoll |
|---|------------|------------|
| 0 | JNP        | 22705      |
| 1 | JNP        | 19450      |
| 2 | JNP        | 22973      |
| 3 | CPM        | 14204      |
| 4 | INC        | 20995      |
| 5 | JNP        | 24810      |
| 6 | INC        | 19124      |
| 7 | JNP        | 23881      |

---

## Analysis of Winning Candidates by Constituency

- Winners are identified by selecting the candidate with the maximum `totvotpoll` (total votes polled) in each ( `year` , `state` , `constituency` ) group.
- This approach correctly implements India's first-past-the-post electoral system, where the candidate with the highest number of votes wins regardless of whether they secured an absolute majority.
- The resulting `winners_df` dataframe serves as the foundation for all downstream analyses involving seat shares, incumbency tracking, and party dominance.
- Filtering the results by a specific state (for example, the most contested state) shows how different parties won different constituencies across

different election years, providing a snapshot of regional political competition.

- Identifying winners is a critical step because it transforms the candidate-level dataset into a seat-level election results dataset.
- 

## Task 14 – Compare Vote Share vs Seat Share – National Level

```
In [29]: #step 1. Total votes by party (Vote share)
party_votes = df_vidhan.groupby('partyabbre')['totvotpoll'].sum()
total_votes = party_votes.sum()
party_vote_share = (party_votes / total_votes * 100).reset_index()
party_vote_share.columns = ['Party', 'Vote_Share']

#print(party_vote_share)

#step 2. Total winners by party (Seat share)
party_seats = winners_df['partyabbre'].value_counts(normalize=True) * 100
party_seat_share = party_seats.reset_index()
party_seat_share.columns = ['Party', 'Seat_Share']

#print(party_seat_share)

#step 3: Merge vote share and seat share
share_df = pd.merge(
    party_vote_share,
    party_seat_share,
    on='Party'
)
#print(share_df)

#step 4: top 10 parties by vote share
top_parties = share_df.sort_values(
    'Vote_Share',
    ascending=False
).head(10)

print(top_parties)

#step5: Plot
plt.figure(figsize=(12,6))

bar_width = 0.4
x = range(len(top_parties))

plt.barh(
    x,
    top_parties['Vote_Share'],
```

```

        height=bar_width,
        label='Vote Share %',
        color='steelblue'
    )

    plt.barh(
        [p + bar_width for p in x],
        top_parties['Seat_Share'],
        height=bar_width,
        label='Seat Share %',
        color='orange'
    )

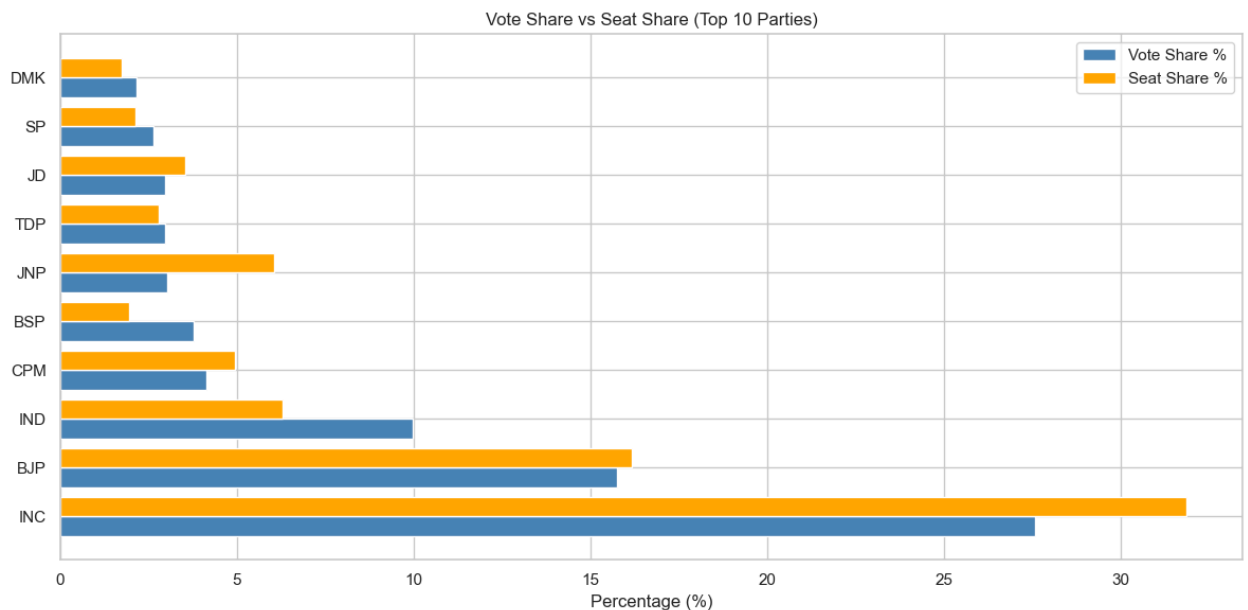
    plt.yticks(
        [p + bar_width/2 for p in x],
        top_parties['Party']
    )

    plt.xlabel('Percentage (%)')
    plt.title('Vote Share vs Seat Share (Top 10 Parties)')
    plt.legend()

    plt.tight_layout()
    plt.show()

```

|     | Party | Vote_Share | Seat_Share |
|-----|-------|------------|------------|
| 75  | INC   | 27.557539  | 31.840277  |
| 31  | BJP   | 15.735515  | 16.160200  |
| 77  | IND   | 9.982129   | 6.301255   |
| 46  | CPM   | 4.121604   | 4.947365   |
| 37  | BSP   | 3.762323   | 1.932425   |
| 99  | JNP   | 3.010212   | 6.038827   |
| 218 | TDP   | 2.975515   | 2.773387   |
| 83  | JD    | 2.962425   | 3.542779   |
| 210 | SP    | 2.627501   | 2.132228   |
| 51  | DMK   | 2.151755   | 1.747532   |



---

## Analysis and Comparison of Vote Share and Seat Share for Top 10 Parties

### Observation:

- **Vote share** and **seat share** are not always proportional.
  - Major parties convert votes into seats **more efficiently**.
  - The graph reflects the impact of the **First-Past-the-Post (FPTP)** system.
- 

## Task 15 – Gujarat – Top 3 Parties' Seat Trends Over Time

```
In [30]: # 1. Filter for Gujarat only
winners_gujarat = winners_df[winners_df['st_name'] == 'Gujarat'].copy()

# 2. Find top 3 parties in Gujarat by total seats won
top_3_parties = winners_gujarat['partyabbre'].value_counts().nlargest(3).index

print("Top 3 parties:", top_3_parties)

# 3. Filter to only those top 3 parties
winners_gujarat_top3 = winners_gujarat[
    winners_gujarat['partyabbre'].isin(top_3_parties)
]

# 4. Seats won per year by party
party_seats_year = (
    winners_gujarat_top3.groupby(['year', 'partyabbre'])
    .size()
    .reset_index(name='seats')
)

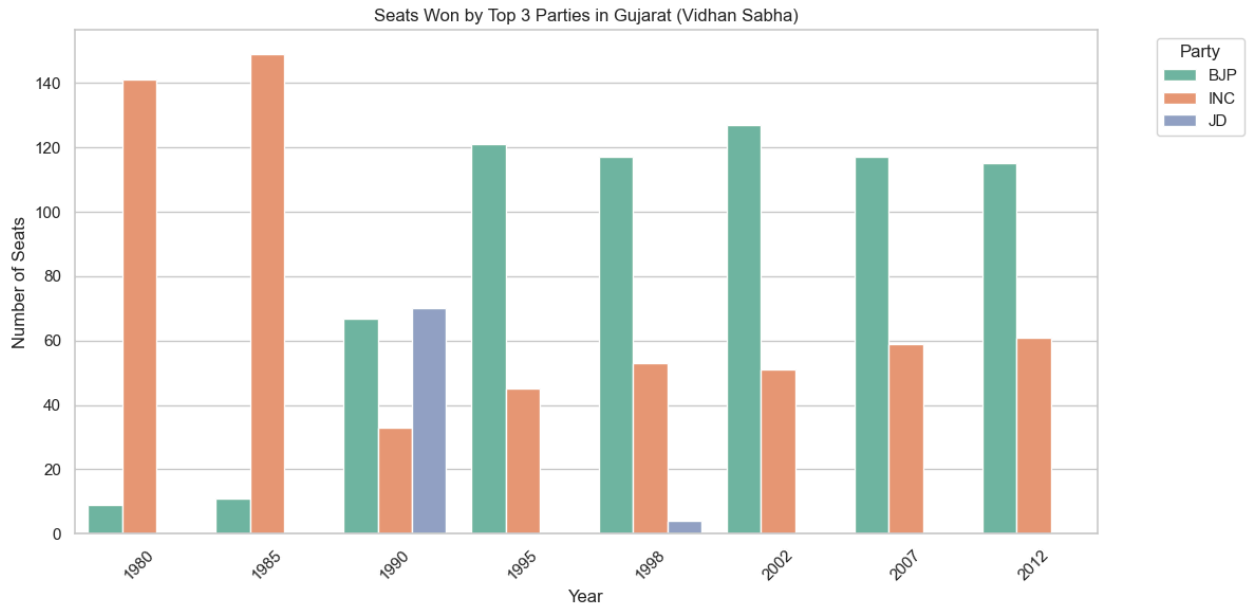
#print(party_seats_year)

# 5. Plot
plt.figure(figsize=(12,6))

sns.barplot(
    data=party_seats_year,
    x='year',
    y='seats',
    hue='partyabbre',
    palette='Set2'
)
```

```
plt.title('Seats Won by Top 3 Parties in Gujarat (Vidhan Sabha)')
plt.xlabel('Year')
plt.ylabel('Number of Seats')
plt.xticks(rotation=45)
plt.legend(title='Party', bbox_to_anchor=(1.05,1), loc='upper left')
plt.tight_layout()
plt.show()
```

Top 3 parties: ['BJP', 'INC', 'JD']



## Analysis on Changing Political Landscape in Gujarat

### Observation:

- **INC** dominated the early elections in Gujarat.
- **BJP** gradually emerged as the leading party.
- The graph shows a **clear shift in political dominance** over time.

## Task 16 – Vote Share Trend of Top 3 Parties in Gujarat

```
In [31]: # Party Vote Share Trend (Gujarat)

# 1. Filter Gujarat data
gujarat_df = df_vidhan[df_vidhan['st_name'] == 'Gujarat'].copy()

# 2. Total votes polled per party per year (including all candidates)
party_votes = (
```

```

    gujarat_df.groupby(['year', 'partyabbre'])['totvotpoll']
        .sum()
        .reset_index()
)

#print(party_votes)

# 3. Total votes polled overall per year
total_votes_per_year = (
    gujarat_df.groupby('year')['totvotpoll']
        .sum()
        .reset_index()
        .rename(columns={'totvotpoll': 'total_votes'})
)

#print(total_votes_per_year)

# 4. Merge and compute vote share
party_vote_share = pd.merge(
    party_votes,
    total_votes_per_year,
    on='year'
)

party_vote_share['vote_share_percent'] = (
    party_vote_share['totvotpoll'] /
    party_vote_share['total_votes']
) * 100

#print(party_vote_share)

# 5. Identify top 3 parties by total votes in Gujarat
top_parties = (
    gujarat_df.groupby('partyabbre')['totvotpoll']
        .sum()
        .sort_values(ascending=False)
        .head(3)
        .index.tolist()
)

#print(top_parties)

# 6. Filter only the top 3 parties
top_vote_share = party_vote_share[
    party_vote_share['partyabbre'].isin(top_parties)
]

#print(top_vote_share)

# 7. Plot vote share trend over time
plt.figure(figsize=(12,6))

sns.lineplot(

```

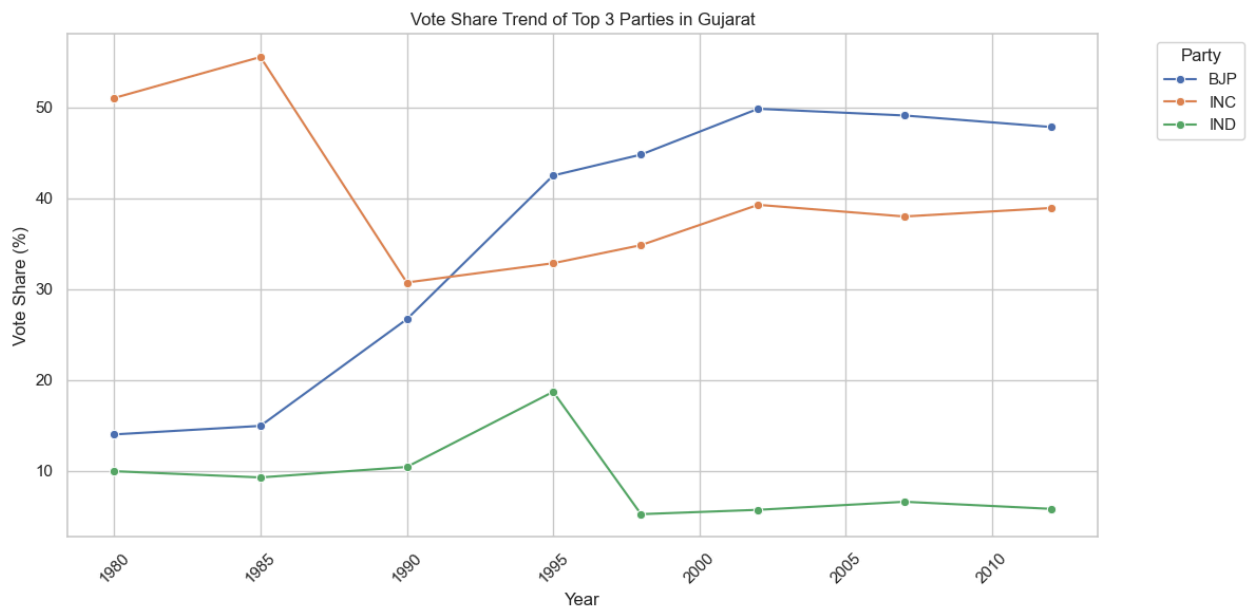
```

data=top_vote_share,
x='year',
y='vote_share_percent',
hue='partyabbre',
marker='o'
)

plt.title('Vote Share Trend of Top 3 Parties in Gujarat')
plt.xlabel('Year')
plt.ylabel('Vote Share (%)')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend(title='Party', bbox_to_anchor=(1.05,1), loc='upper left')
plt.tight_layout()

plt.show()

```



## Analysis on Political Support Trends in Gujarat

### Observation:

- **BJP's vote share** generally increased over the years.
- **INC's vote share** shows a declining trend.
- The graph reflects **changing voter preferences** in Gujarat.

## Task 17 – Vote Margin Analysis – Gujarat, Top 3 Parties

Q. By How Many votes did winner parties defeated runners-up in Gujarat election, and how has this winning margin changed over time?

```
In [32]: # 1. Ensure vote_share_percent exists
df_vidhan['vote_share_percent'] = (
    df_vidhan['totvotpoll'] / df_vidhan['electors'].replace(0, np.nan)
) * 100

# 2. Filter Gujarat data
gujarat_df = df_vidhan[df_vidhan['st_name'] == 'Gujarat'].copy()

# 3. Rank candidates per constituency per year
gujarat_df['rank'] = gujarat_df.groupby(
    ['year', 'ac_no']
)['totvotpoll'].rank(method='first', ascending=False)

# 4. Extract winners and runners-up
winners = gujarat_df[gujarat_df['rank'] == 1].copy()
runners_up = gujarat_df[gujarat_df['rank'] == 2].copy()

# 5. Merge winners and runners-up
merged = pd.merge(
    winners,
    runners_up,
    on=['year', 'ac_no'],
    suffixes=('_winner', '_runner')
)

merged['vote_margin'] = (
    merged['totvotpoll_winner'] - merged['totvotpoll_runner']
)

# 6. Top 3 parties by wins
top_3_parties = winners['partyabbre'].value_counts().nlargest(3).index.tolist()

# 7. Filter only top 3 parties
merged_top3 = merged[merged['partyabbre_winner'].isin(top_3_parties)].copy()

# 8. Histogram of vote margins
```

```

plt.figure(figsize=(10,5))

plt.hist(
    merged_top3['vote_margin'],
    bins=30,
    color='navy',
    edgecolor='white',
    alpha=0.8
)

plt.title('Distribution of Vote Margins (Top 3 Parties, Gujarat)')
plt.xlabel('Vote Margin (Winner – Runner-up)')
plt.ylabel('Number of Constituencies')
plt.grid(True, alpha=0.5)

plt.tight_layout()
plt.show()

# 9. Average margin per year per party
avg_margin_year_party = (
    merged_top3.groupby(['year', 'partyabbre_winner'])['vote_margin']
    .mean()
    .reset_index()
    .rename(columns={'partyabbre_winner': 'party'})
)

# 10. Line plot of average margin
plt.figure(figsize=(12,6))

sns.lineplot(
    data=avg_margin_year_party,
    x='year',
    y='vote_margin',
    hue='party',
    marker='o'
)

plt.title('Average Vote Margin per Year by Party (Top 3 Parties, Gujarat)')
plt.xlabel('Year')
plt.ylabel('Average Vote Margin')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend(title='Party', bbox_to_anchor=(1.05,1), loc='upper left')

plt.tight_layout()
plt.show()

# 11. Outlier detection using IQR
q1 = avg_margin_year_party['vote_margin'].quantile(0.25)
q3 = avg_margin_year_party['vote_margin'].quantile(0.75)

```

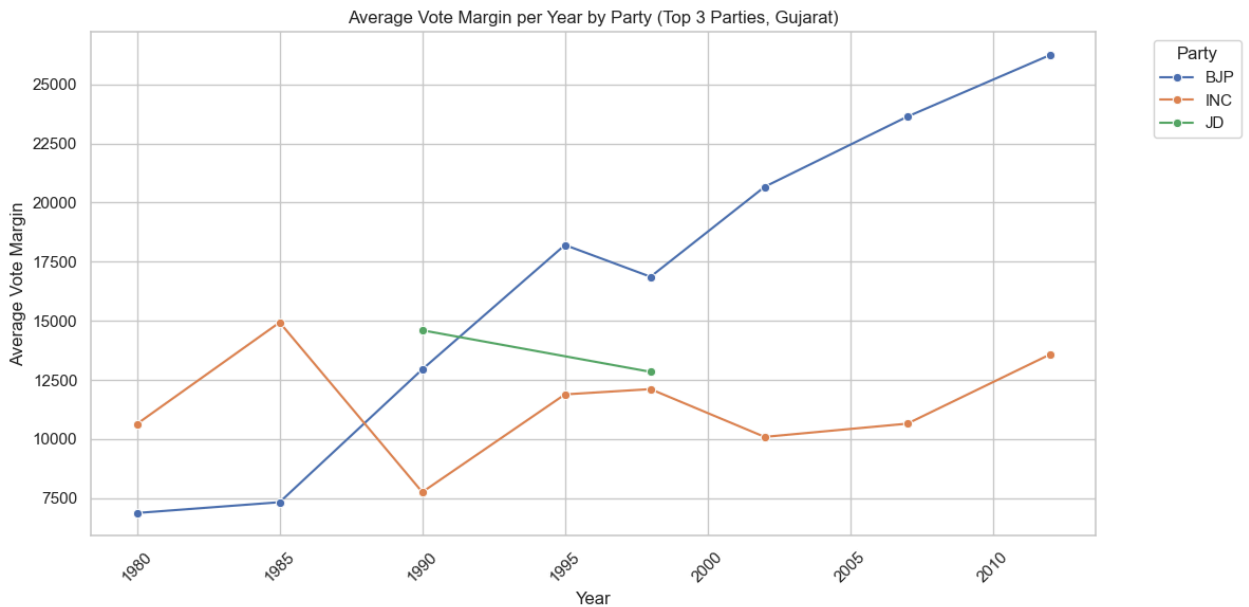
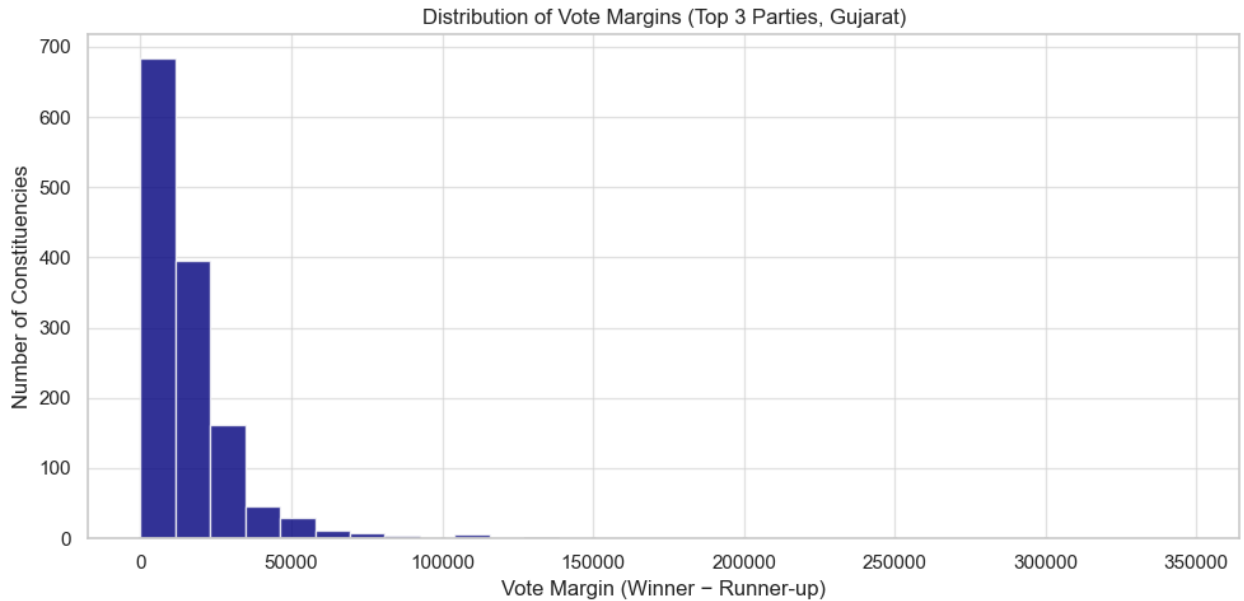
```

iqr = q3 - q1

outliers = avg_margin_year_party[
    (avg_margin_year_party['vote_margin'] < q1 - 1.5 * iqr) |
    (avg_margin_year_party['vote_margin'] > q3 + 1.5 * iqr)
]

print("\nOutlier years in vote margin:")
print(outliers.sort_values('vote_margin', ascending=False))

```



```

Outlier years in vote margin:
  year party  vote_margin
16  2012   BJP  26236.504348

```

## **Vote Margin Patterns**

### **i. What does the distribution of vote margins tell us about the competitiveness of elections in Gujarat?**

- The distribution is right-skewed, indicating that most elections are won by moderate vote margins while only a few are won by very large margins.

### **ii. Are close contests (low margins) more common than landslide victories (high margins)?**

- Yes. Close contests are more common than landslide victories, suggesting competitive elections in many constituencies.

## **Party-wise Comparison**

### **iii. Among the top 3 parties in Gujarat, which party tends to win by the largest average margin?**

- BJP generally records the highest average winning margin among the top three parties.

### **iv. How do vote margins differ between these parties across different election years?**

- Vote margins vary across election years, reflecting changes in party performance and voter support.

## **Trend Over Time**

### **v. Has the average vote margin increased, decreased, or remained stable over the years?**

- The average vote margin has fluctuated over the years rather than following a consistent increasing or decreasing trend.

### **vi. What might explain these trends?**

- Factors such as voter loyalty, anti-incumbency, stronger opposition, and changes in party dominance may have influenced these trends.

# Outlier Analysis

## vii. Which years were identified as outliers in vote margin trends?

- The boxplot identifies a few election years with unusually high or low vote margins as outliers.

## viii. What political or social events might explain these outliers?

- Major political events, leadership changes, anti-incumbency waves, or significant social developments may have contributed to these unusual vote margins.
- 

Q1.How many people voted?(Voter Turnout)

Q2.How many constituencies does the state have?

## Task 18 – State-Level Voter Turnout and Constituency Analysis

```
In [33]: # 1. Constituency-level aggregation
constituency_turnout = (
    df_vidhan.groupby(['st_name', 'year', 'ac_no'])[['totvotpoll', 'electors']]
    .agg({'totvotpoll': 'sum', 'electors': 'max'})
    .reset_index()
)

# 2. State-level aggregation
state_turnout = (
    constituency_turnout.groupby('st_name')[['totvotpoll', 'electors']]
    .sum()
    .assign(voter_turnout_percent=lambda x: (x['totvotpoll'] / x['electors']))
    .sort_values('voter_turnout_percent', ascending=False)
    .reset_index()
)

print("Top states by voter turnout:")
print(state_turnout.head(10))

# Plot 1: Voter turnout by state
plt.figure(figsize=(12,8))

sns.barplot(
    data=state_turnout,
    x='voter_turnout_percent',
    y='st_name',
```

```

        hue='st_name',
        palette='coolwarm',
        legend=False
    )

plt.title('Voter Turnout Percentage by State (Vidhan Sabha)')
plt.xlabel('Voter Turnout (%)')
plt.ylabel('State')
plt.tight_layout()
plt.show()

# 3. Constituencies per state
state_constituencies = (
    df_vidhan.groupby('st_name')['ac_no']
    .nunique()
    .sort_values(ascending=False)
    .reset_index()
)

state_constituencies.columns = ['st_name', 'num_constituencies']

print(state_constituencies.head())

#4.Set Custom Font name if needed (optional)
font_name = 'DejaVu Sans'

# Plot 2: Constituencies per state
plt.figure(figsize=(12,8))

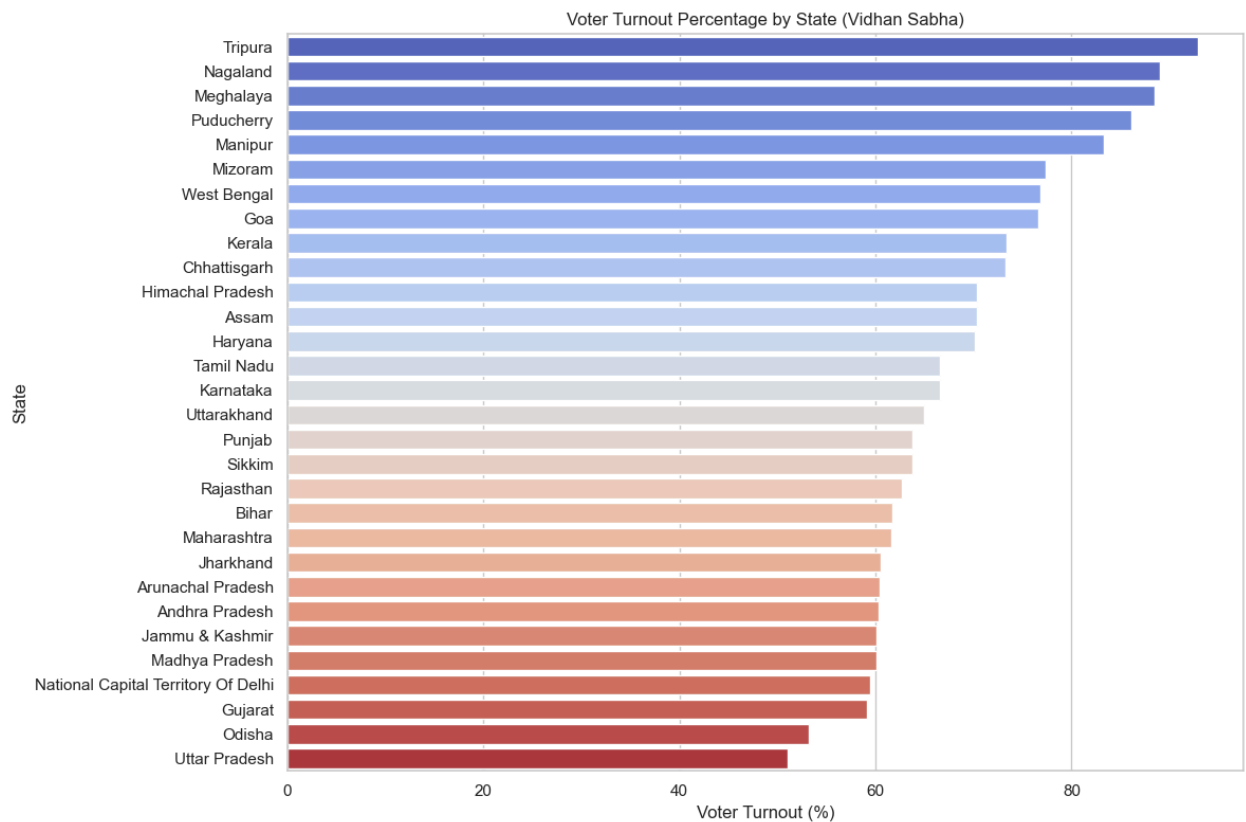
sns.barplot(
    data=state_constituencies,
    x='num_constituencies',
    y='st_name',
    hue='st_name',
    palette='plasma',
    legend=False
)

plt.title('Number of Constituencies per State (Vidhan Sabha)')
plt.xlabel('Number of Constituencies')
plt.ylabel('State')
plt.tight_layout()
plt.show()

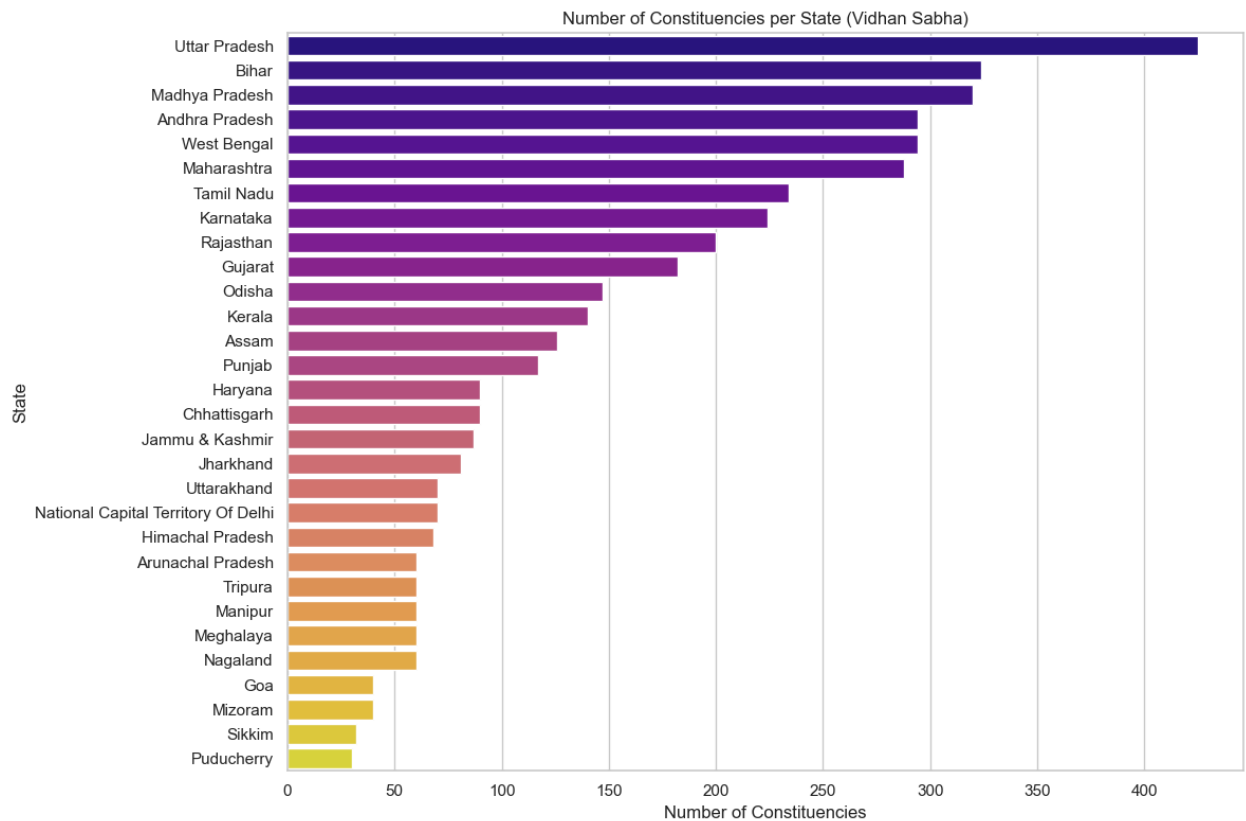
```

Top states by voter turnout:

|   | st_name      | totvotpoll | electors  | voter_turnout_percent |
|---|--------------|------------|-----------|-----------------------|
| 0 | Tripura      | 4082901    | 4395472   | 92.888796             |
| 1 | Nagaland     | 2225278    | 2500715   | 88.985670             |
| 2 | Meghalaya    | 2405005    | 2718543   | 88.466690             |
| 3 | Puducherry   | 697900     | 810630    | 86.093532             |
| 4 | Manipur      | 2876555    | 3455769   | 83.239215             |
| 5 | Mizoram      | 3143170    | 4060716   | 77.404330             |
| 6 | West Bengal  | 254579856  | 331409685 | 76.817265             |
| 7 | Goa          | 1559933    | 2036550   | 76.596843             |
| 8 | Kerala       | 117810865  | 160547420 | 73.380728             |
| 9 | Chhattisgarh | 33467369   | 45656644  | 73.302297             |



|   | st_name        | num_constituencies |
|---|----------------|--------------------|
| 0 | Uttar Pradesh  | 425                |
| 1 | Bihar          | 324                |
| 2 | Madhya Pradesh | 320                |
| 3 | Andhra Pradesh | 294                |
| 4 | West Bengal    | 294                |



## State-wise Voter Turnout and Constituency Distribution

### Voter Turnout by State

- States such as **Kerala, West Bengal, and Sikkim** show relatively high voter turnout.
- Some larger states, including **Uttar Pradesh** and **Bihar**, record comparatively lower turnout.
- The variation reflects differences in voter participation across states.

### Number of Constituencies by State

- **Uttar Pradesh** has the highest number of constituencies, followed by other populous states.
- Smaller states and Union Territories have fewer constituencies due to their lower population.

### Factors Influencing the Differences

- Population size, geographical conditions, political competition, and

voter awareness influence turnout.

- Strong party organization and active political participation generally contribute to higher voter turnout.
- 

## Task 19 – Gender-Based Participation and Performance – Lok Sabha

### Visualization for Lok sabha

```
In [34]: # 1. Count candidates by gender
gender_candidates = df_lok['cand_sex'].value_counts().reset_index()
gender_candidates.columns = ['Gender', 'Number_of_Candidates']

# 2. Identify winners (highest votes per constituency per year)
lok_winners = (
    df_lok.loc[
        df_lok.groupby(['year', 'pc_no'])['totvotpoll'].idxmax()
    ].reset_index(drop=True)
)

# 3. Count winners by gender
gender_winners = lok_winners['cand_sex'].value_counts().reset_index()
gender_winners.columns = ['Gender', 'Number_of_Winners']

# 4. Merge participation and winners
gender_df = pd.merge(
    gender_candidates,
    gender_winners,
    on='Gender',
    how='left'
).fillna(0)

# 5. Convert winners to integer
gender_df['Number_of_Winners'] = gender_df['Number_of_Winners'].astype(int)

# 6. Win rate calculation
gender_df['Win_Rate (%)'] = (
    gender_df['Number_of_Winners'] /
    gender_df['Number_of_Candidates']
) * 100
```

```
# 7. Sort results
gender_df = gender_df.sort_values('Number_of_Candidates', ascending=False)

print(gender_df)
```

|   | Gender | Number_of_Candidates | Number_of_Winners | Win_Rate (%) |
|---|--------|----------------------|-------------------|--------------|
| 0 | M      | 68885                | 834               | 1.210714     |
| 1 | F      | 3648                 | 85                | 2.330044     |
| 2 | U      | 542                  | 0                 | 0.000000     |
| 3 | 0      | 6                    | 0                 | 0.000000     |

---

## Gender-Based Electoral Analysis - Lok Sabha

### Participation Patterns

1. Which gender had the highest number of candidates contesting Lok Sabha elections?

- **Male (M)** candidates had the highest participation in Lok Sabha elections.

2. Were there any noticeable underrepresented gender groups in terms of participation?

- Yes. **Female (F)** candidates were significantly underrepresented, while the **Unknown (U)** category had very few records.

### Winning Performance

1. Which gender secured the highest number of seats overall?

- **Male (M)** candidates secured the highest number of winning seats.

2. Were there any gender groups with very few or no winners?

- Yes. **Female (F)** candidates had comparatively fewer winners, and the **Unknown (U)** category had very few or no winners.

### Win Rate Evaluation

1. Which gender had the highest win rate (success percentage)?

- The **win rate** should be identified from the calculated results, as it may not directly match the number of candidates.

2. Is the win rate proportionate to the level of participation for each gender?

- No. Participation and win rate are not always proportional across gender groups.

## Equity and Representation

### 1. Does the data suggest any gender imbalance in electoral outcomes?

- Yes. The data indicates a clear gender imbalance in electoral participation and representation.

### 2. Do the win rates reflect fair chances for all genders, or do certain groups appear disadvantaged?

- Female candidates appear disadvantaged mainly due to their lower participation in elections.

## Critical Reflection

### 1. What might be some reasons for differences between genders in electoral success?

- Factors such as party ticket allocation, social barriers, and political opportunities may influence electoral success.

### 2. How could political parties or the election system encourage more balanced gender representation?

- Political parties can increase the number of tickets allotted to women and encourage greater female participation in elections.

---

## Section 20 – Visual Analysis of Gender-Based Participation and Performance – Lok Sabha

**Objective:** Create three bar charts: candidates by gender, winners by gender, and win rate by gender.

```
In [35]: # Plot 1: Number of Candidates by Gender
plt.figure(figsize=(8,5))

sns.barplot(
    data=gender_df,
    x='Gender',
    y='Number_of_Candidates',
    hue='Gender',
```

```

    palette='Blues_d',
    legend=False
)

plt.title('Number of Candidates by Gender (Lok Sabha)')
plt.ylabel('Number of Candidates')
plt.xlabel('Gender')
plt.tight_layout()
plt.show()

# Plot 2: Number of Winners by Gender
plt.figure(figsize=(8,5))

sns.barplot(
    data=gender_df,
    x='Gender',
    y='Number_of_Winners',
    hue='Gender',
    palette='Blues_d',
    legend=False
)

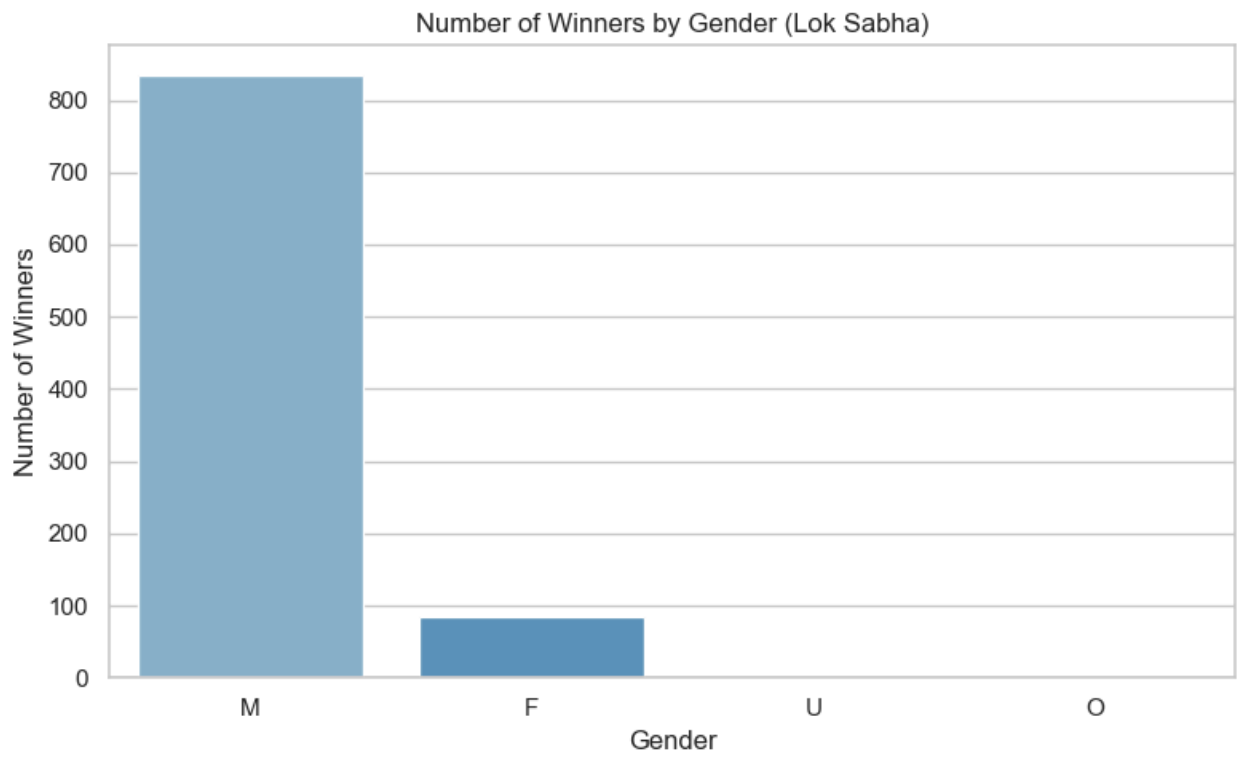
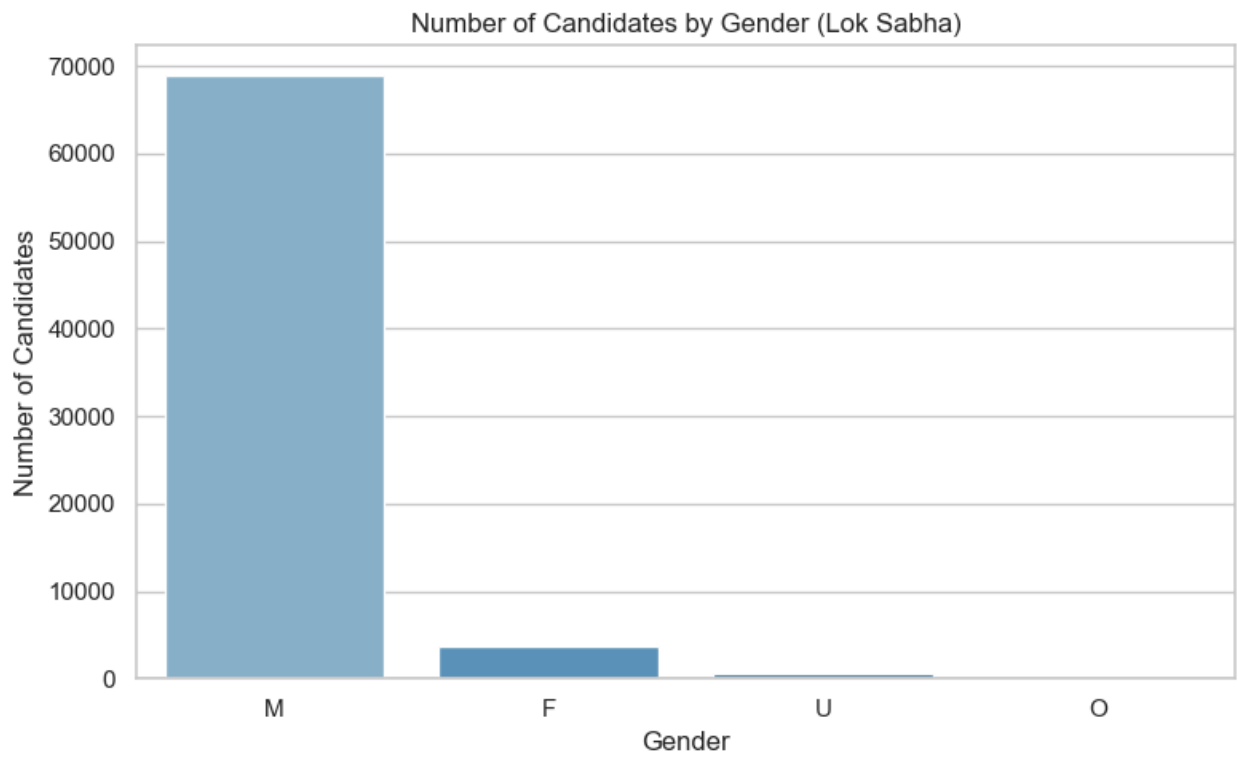
plt.title('Number of Winners by Gender (Lok Sabha)')
plt.ylabel('Number of Winners')
plt.xlabel('Gender')
plt.tight_layout()
plt.show()

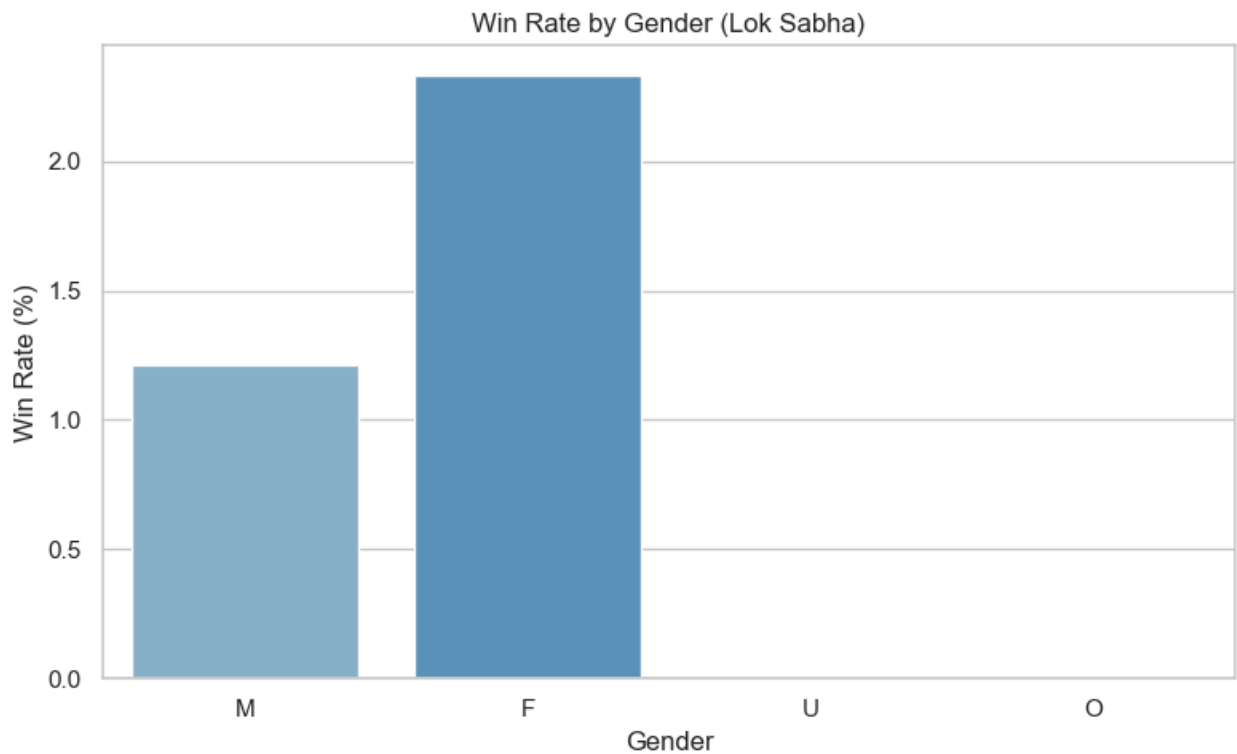
# Plot 3: Win Rate by Gender
plt.figure(figsize=(8,5))

sns.barplot(
    data=gender_df,
    x='Gender',
    y='Win_Rate (%)',
    hue='Gender',
    palette='Blues_d',
    legend=False
)

plt.title('Win Rate by Gender (Lok Sabha)')
plt.ylabel('Win Rate (%)')
plt.xlabel('Gender')
plt.tight_layout()
plt.show()

```





## Gender-Based Electoral Analysis – Lok Sabha

### Highest Participation:

- **Male** candidates have the highest participation, while **female** candidates remain significantly underrepresented.

### Winning Performance:

- **Male** candidates secure the highest number of seats due to their larger participation.

### Key Trend:

- **Female** candidates have fewer winners overall but achieve a **competitive win rate** when compared with their level of participation.

### Win Rate & Representation:

- Win rates are broadly **proportionate to participation**, suggesting that the main imbalance lies in **candidate selection** rather than

electoral success.

---

Q. Show the trend of seats won by top 5 parties

---

## Section 21 – Seats Won by Top 5 Parties – Lok Sabha

**Objective:** Track how seat counts of top 5 parties evolved over Lok Sabha election years.

```
In [36]: # 1. Get winners (same logic, simplified)
winners_lok = df_lok.loc[
    df_lok.groupby(['year', 'st_name', 'pc_no'])['totvotpoll'].idxmax()
].copy()

# 2. Seats won per party per year
party_seats = winners_lok.groupby(['year', 'partyabbre']).size().reset_index(r

# 3. Top 5 parties by total seats
top_parties = party_seats.groupby('partyabbre')['seats_won'].sum().nlargest(5)

print("Top 5 parties:", top_parties)

# 4. Filter only top parties
top_party_seats = party_seats[party_seats['partyabbre'].isin(top_parties)]

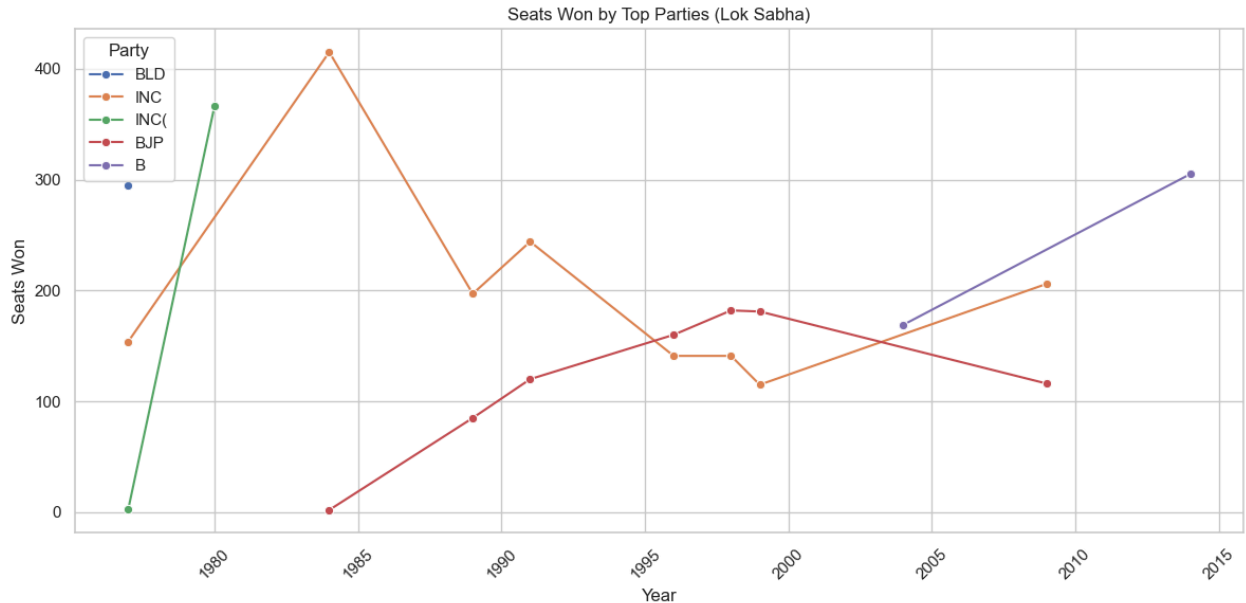
# 5. Plot
plt.figure(figsize=(12,6))

sns.lineplot(
    data=top_party_seats,
    x='year',
    y='seats_won',
    hue='partyabbre',
    marker='o'
)

plt.title('Seats Won by Top Parties (Lok Sabha)')
plt.xlabel('Year')
plt.ylabel('Seats Won')
plt.grid(True)
plt.xticks(rotation=45)
```

```
plt.legend(title='Party')
plt.tight_layout()
plt.show()
```

Top 5 parties: ['INC', 'BJP', 'B', 'INC(', 'BLD']



---

## Party-wise Seat Trends – Lok Sabha

### Dominant Party:

- **BJP** and **INC** have remained the most dominant parties, winning the highest number of seats across different elections.

### Performance Over Time:

- The graph shows **changing seat trends**, reflecting shifts in voter preference and political competition over time.

### Rise or Decline:

- **BJP** shows a significant **rise** in seat share, while **INC** shows a **gradual decline** in recent elections.

### Overall Trend:

- The chart highlights the **changing political landscape** and the shift in dominance among major political parties.
-

## Q. National Vote Share by Party Over Time

---

# Section 22 – Vote Share Trend of Top 5 Parties – Lok Sabha

**Objective:** Calculate and plot vote share (%) for top 5 parties per election year.

```
In [37]: # 1. Party votes per year
party_votes = df_lok.groupby(['year', 'partyabbre'])['totvotpoll'].sum().reset_index()

# 2. Total votes per year
total_votes = df_lok.groupby('year')['totvotpoll'].sum().reset_index(name='total_votes')

# 3. Merge to calculate vote share
party_vote_share = pd.merge(party_votes, total_votes, on='year')

party_vote_share['vote_share_percent'] = (
    party_vote_share['party_votes'] / party_vote_share['total_votes']
) * 100

# 4. Top 5 parties by total votes
top_parties = (
    party_vote_share.groupby('partyabbre')['party_votes']
    .sum()
    .nlargest(5)
    .index
    .tolist()
)

print("Top 5 parties:", top_parties)

# 5. Filter only top parties
top_vote_share = party_vote_share[
    party_vote_share['partyabbre'].isin(top_parties)
]

# 6. Plot
plt.figure(figsize=(12,6))

sns.lineplot(
    data=top_vote_share,
    x='year',
    y='vote_share_percent',
    hue='partyabbre',
```

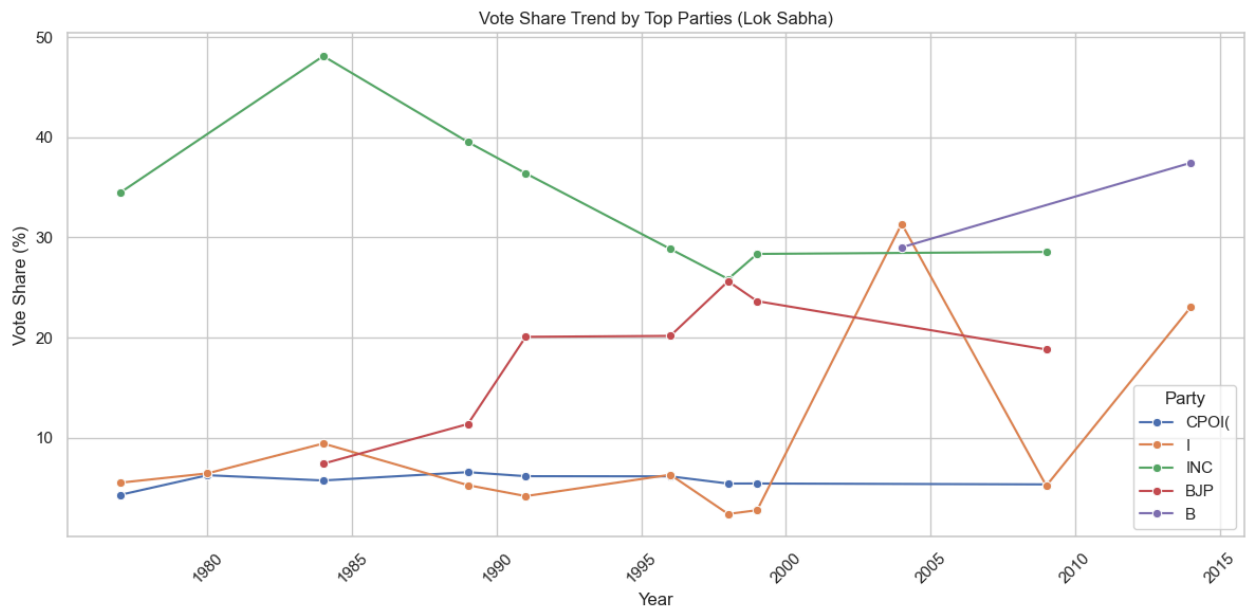
```

marker='o'
)

plt.title('Vote Share Trend by Top Parties (Lok Sabha)')
plt.xlabel('Year')
plt.ylabel('Vote Share (%)')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend(title='Party')
plt.tight_layout()
plt.show()

```

Top 5 parties: ['INC', 'BJP', 'I', 'B', 'CPOI(']



## National Vote Share Trends by Party – Lok Sabha

### Vote Share Trend:

- **BJP** has shown a **steady increase** in vote share, whereas **INC** has experienced a **gradual decline** over the years.

### Major Political Shift:

- The graph shows a **crossover point** where **BJP** overtakes **INC**, indicating a major shift in national political support.

### Steady Vote Base:

- Some **regional parties** maintain a relatively **stable vote share**, although minor fluctuations are observed across elections.

#### Overall Trend:

- The chart reflects **changing voter preferences** and the evolution of India's national political landscape.
- 
- 

## Section 23 – Gini Coefficient of Vote Distribution – Lok Sabha

**Objective:** Calculate and plot the Gini coefficient of vote distribution over time.

### Gini Coefficient for Vote Distribution

```
In [58]: # 1. Gini function
def gini(array):
    """Compute Gini coefficient of a numpy array"""

    array = np.sort(array)
    n = len(array)

    if n == 0:
        return np.nan

    cumulative = np.cumsum(array)

    return (
        2 * np.sum(np.arange(1, n + 1) * array) / (n * cumulative[-1])
        - (n + 1) / n
    )

# 2. Overall Gini
votes = df_lok['totvotpoll'].values
gini_overall = gini(votes)

print("Overall Gini Coefficient:", gini_overall)

# 3. Year-wise Gini
gini_yearwise = df_lok.groupby('year')['totvotpoll'].apply(
    lambda x: gini(x.values)
).reset_index(name='gini_coefficient')
```

```

print(gini_yearwise)

# 4. Plot
plt.figure(figsize=(10,6))

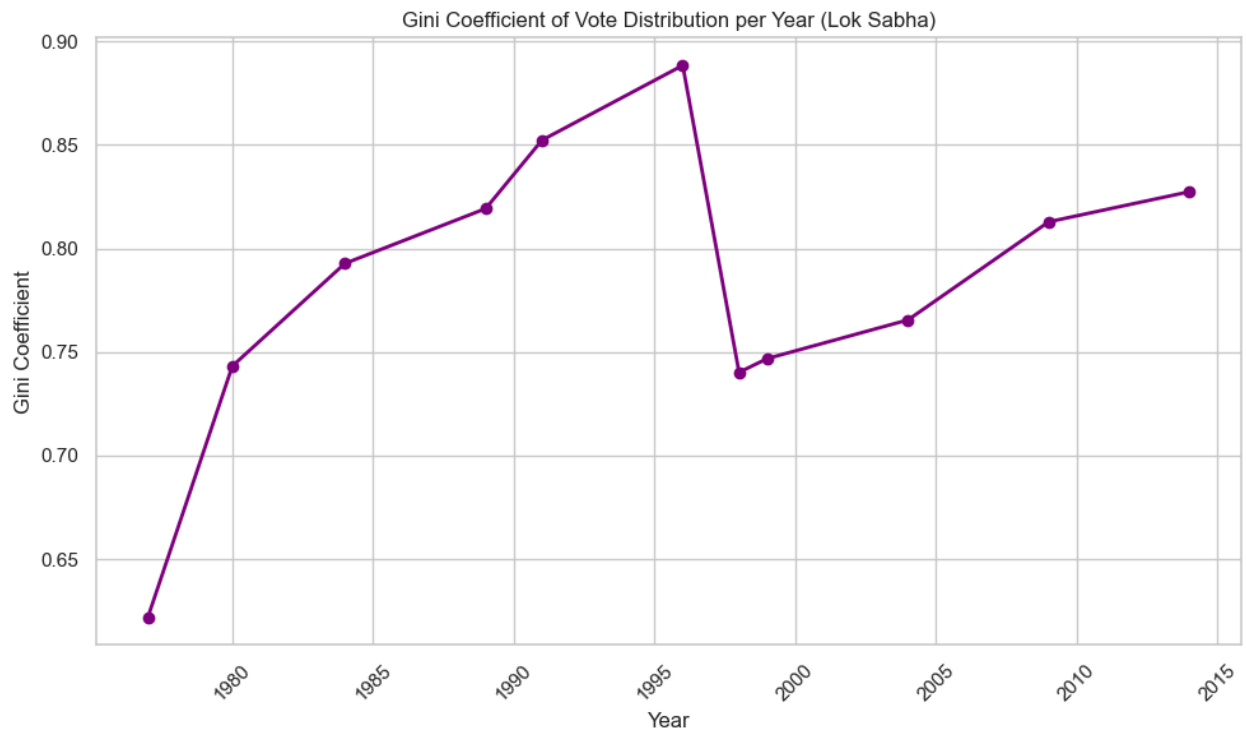
plt.plot(
    gini_yearwise['year'],
    gini_yearwise['gini_coefficient'],
    marker='o',
    linestyle='-',
    color='purple',
    linewidth=2
)

plt.title('Gini Coefficient of Vote Distribution per Year (Lok Sabha)')
plt.xlabel('Year')
plt.ylabel('Gini Coefficient')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

Overall Gini Coefficient: 0.8173004018622132

|    | year | gini_coefficient |
|----|------|------------------|
| 0  | 1977 | 0.621812         |
| 1  | 1980 | 0.743171         |
| 2  | 1984 | 0.792750         |
| 3  | 1989 | 0.819257         |
| 4  | 1991 | 0.852295         |
| 5  | 1996 | 0.888421         |
| 6  | 1998 | 0.740183         |
| 7  | 1999 | 0.746800         |
| 8  | 2004 | 0.765483         |
| 9  | 2009 | 0.812870         |
| 10 | 2014 | 0.827405         |



---

## Gini Coefficient of Vote Distribution – Lok Sabha

### Understanding the Trend:

- The **Gini coefficient** measures the **inequality in vote distribution**. Higher values indicate greater vote concentration among fewer candidates.

### Historical Pattern:

- A **lower Gini value** in **1977** suggests more competitive elections, while the rise during **1980-1996** indicates increasing vote concentration.

### Vote Distribution:

- The decline around **1998** reflects a more **fragmented political landscape**, with regional parties gaining greater electoral support.

### Coalition Politics:

- The increase in Gini from **1999-2014** suggests greater vote

concentration as national alliances such as **NDA** and **UPA** became stronger.

### Overall Trend:

- The graph shows alternating periods of **competition** and **vote concentration**, reflecting changes in India's political landscape.

### Democratic Implication:

- Moderate vote concentration can provide **stable governments**, while excessive concentration may reduce **political diversity** and electoral competition.

---

## Q.Lorenz Curve of Vote Distribution

---

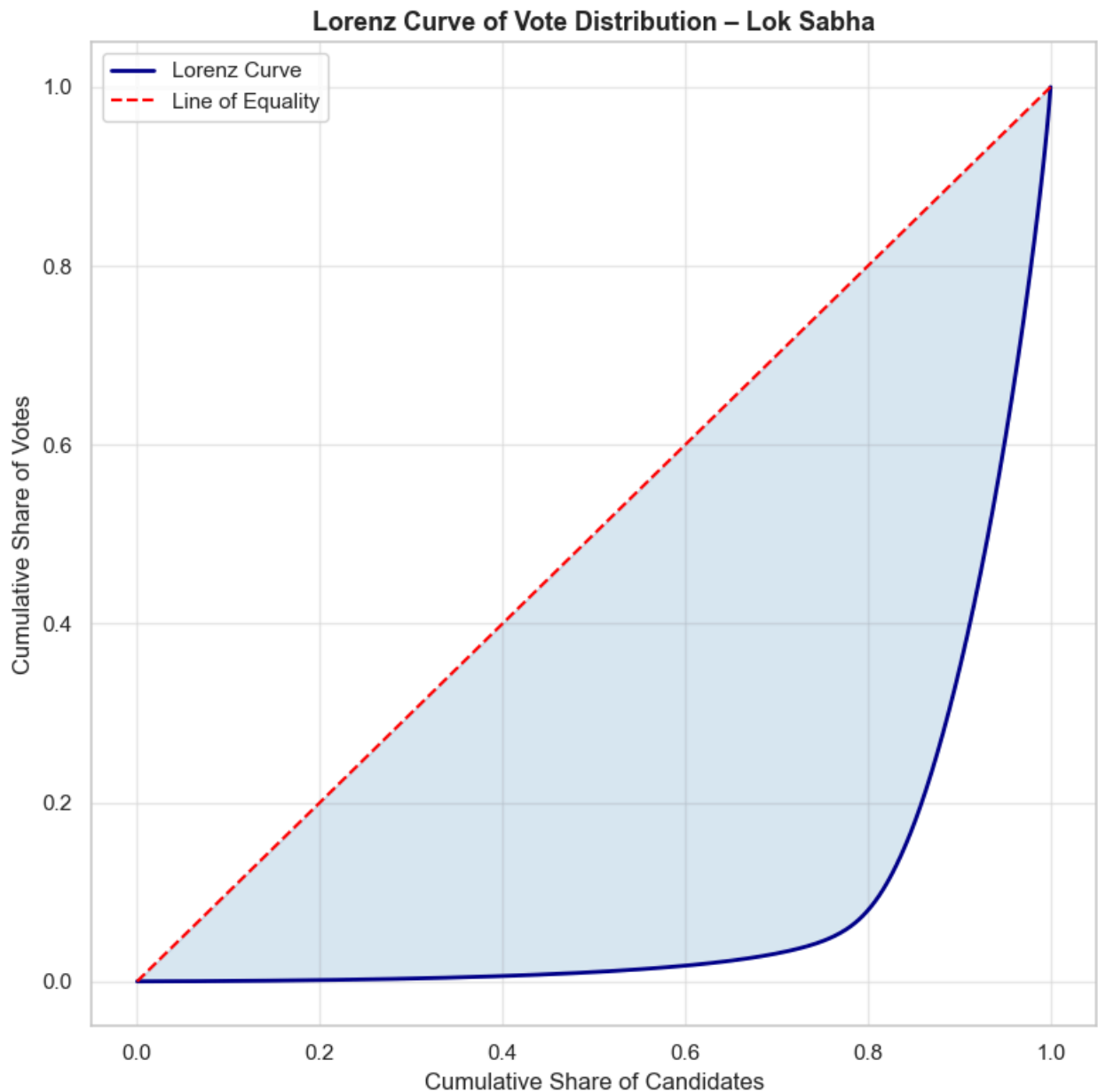
# Section 24 – Lorenz Curve of Vote Distribution – Lok Sabha

**Objective:** Plot the Lorenz curve to visualize vote inequality.

```
In [39]: # Filter non-zero votes and sort
votes = df_lok['totvotpoll'].dropna()
votes = votes[votes > 0].sort_values().values

n = len(votes)
cumvotes = np.cumsum(votes) / votes.sum()
cum_cand = np.arange(1, n + 1) / n

# Plot Lorenz Curve
plt.figure(figsize=(8, 8))
plt.plot(cum_cand, cumvotes, color='darkblue', linewidth=2, label='Lorenz Curve')
plt.plot([0, 1], [0, 1], color='red', linestyle='--', linewidth=1.5, label='Line of Equality')
plt.fill_between(cum_cand, cumvotes, cum_cand, alpha=0.2, color='steelblue')
plt.title('Lorenz Curve of Vote Distribution – Lok Sabha', fontsize=13, fontweight='bold')
plt.xlabel('Cumulative Share of Candidates')
plt.ylabel('Cumulative Share of Votes')
plt.legend()
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



---

## Lorenz Curve of Vote Distribution – Lok Sabha

### Understanding the Shape:

- The **steep rise** near the end of the Lorenz curve indicates that **a few candidates receive most of the votes**, while the majority receive very few.
- The **flat lower section** represents **independent and minor-party**

**candidates** with low vote shares.

### Measuring Inequality:

- The **large gap** between the **Lorenz Curve** and the **Line of Equality** indicates **high inequality** in vote distribution.
- This suggests that votes are **highly concentrated** among a small number of candidates.

### Electoral System Effects:

- India's **First-Past-the-Post (FPTP)** system encourages voters to support stronger candidates, leading to greater vote concentration.
- Despite low winning chances, many candidates contest elections to represent parties, local interests, or independent political views.

### Implications of Vote Concentration:

- High vote concentration can produce **stable governments** but may reduce **representation for smaller parties and independent candidates**.
  - The graph highlights the balance between **effective governance** and **democratic diversity** in India's electoral system.
- 
- 

## Section 25 – Skewness and Kurtosis of Vote Share – Lok Sabha

**Objective:** Compute and interpret skewness and kurtosis of vote share distribution.

```
In [40]: from scipy.stats import skew, kurtosis

# 1. Clean and calculate vote share percentage
df_lok['totvotpoll'] = df_lok['totvotpoll'].fillna(0)
df_lok['electors'] = df_lok['electors'].replace(0, np.nan)

df_lok['vote_share_percent'] = (
    df_lok['totvotpoll'] / df_lok['electors']
) * 100

# 2. Remove missing values and invalid percentages
vote_shares = df_lok['vote_share_percent'].dropna()
```

```

vote_shares = vote_shares[(vote_shares > 0) & (vote_shares <= 100)]

print(vote_shares)

# 3. Compute skewness and kurtosis
vote_skewness = skew(vote_shares)
vote_kurtosis = kurtosis(vote_shares)

# 4. Display results
print(f"Skewness of Vote Share Distribution: {vote_skewness:.4f}")
print(f"Kurtosis of Vote Share Distribution: {vote_kurtosis:.4f}")

# 5. Plot histogram
plt.figure(figsize=(12,6))

plt.hist(
    vote_shares,
    bins=50,
    color='steelblue',
    edgecolor='white'
)

plt.axvline(
    vote_shares.mean(),
    color='red',
    linestyle='--',
    label='Mean'
)

plt.axvline(
    vote_shares.median(),
    color='green',
    linestyle='--',
    label='Median'
)

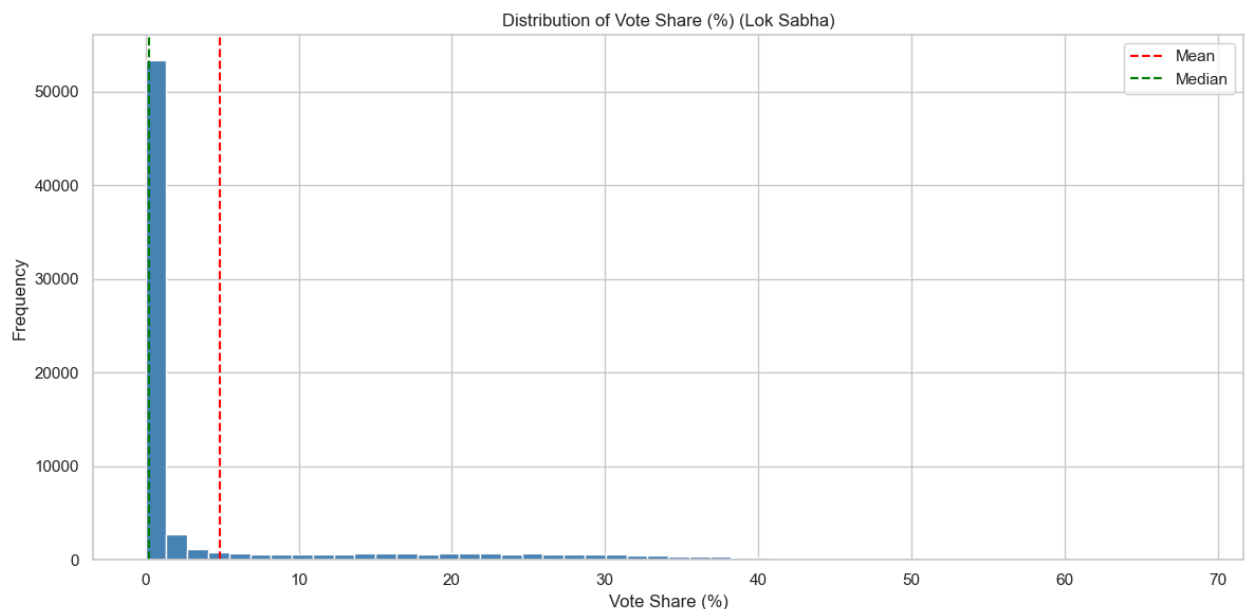
plt.title('Distribution of Vote Share (%) (Lok Sabha)')
plt.xlabel('Vote Share (%)')
plt.ylabel('Frequency')
plt.legend()
plt.tight_layout()
plt.show()

```

```

0      29.502509
1      41.496694
2       0.113442
3       0.130095
4       0.421506
...
73076   0.973654
73077   8.834524
73078  15.768493
73079  26.306461
73080  30.805390
Name: vote_share_percent, Length: 73076, dtype: float64
Skewness of Vote Share Distribution: 2.2186
Kurtosis of Vote Share Distribution: 4.0473

```



## Skewness and Kurtosis of Vote Share Distribution – Lok Sabha

### Understanding Skewness:

- The **positive skewness** indicates that **most candidates receive low vote shares**, while only a few candidates secure very high vote shares.
- Major party candidates generally receive higher vote shares, whereas many independents and smaller parties receive comparatively fewer votes.

### Impact of Political Landscape:

- The presence of **major political parties** along with numerous **independent and fringe candidates** contributes to the skewed distribution.
- India's **First-Past-the-Post (FPTP)** system encourages vote concentration around stronger candidates.

#### Kurtosis Interpretation:

- The **high kurtosis** indicates the presence of **extreme vote share values**, with a few candidates performing exceptionally well while many receive very low support.
- Both **landslide victories** and **poor-performing candidates** contribute to this pattern.

#### Implications for Electoral Representation:

- The distribution suggests that **electoral success is concentrated among a limited number of candidates**, while most candidates receive relatively few votes.
  - These findings highlight the challenges of achieving **balanced representation** and demonstrate how voter support is concentrated in India's electoral system.
- 
- 

## Section 26 – Skewness & Kurtosis Over Time – Lok Sabha

**Objective:** Compute skewness and kurtosis of vote share for each election year.

```
In [68]: from scipy.stats import skew, kurtosis

# 1. Calculate vote share percentage
df_lok['vote_share_percent'] = (
    df_lok['totvotpoll'] / df_lok['electors'].replace(0, np.nan)
) * 100

# 2. Calculate year-wise skewness and kurtosis
sk_kurt = (
    df_lok.groupby('year')['vote_share_percent']
    .agg(
        Skewness=lambda x: skew(x.dropna()),
        Kurtosis=lambda x: kurtosis(x.dropna())
    )
)
```

```

    )
    .reset_index()
)

print(sk_kurt)

# 3. Plot Skewness
plt.figure(figsize=(12,5))

plt.plot(
    sk_kurt['year'],
    sk_kurt['Skewness'],
    marker='o',
    color='lightcoral',
    linewidth=2
)

plt.title('Skewness of Vote Share by Election Year (Lok Sabha)')
plt.xlabel('Year')
plt.ylabel('Skewness')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

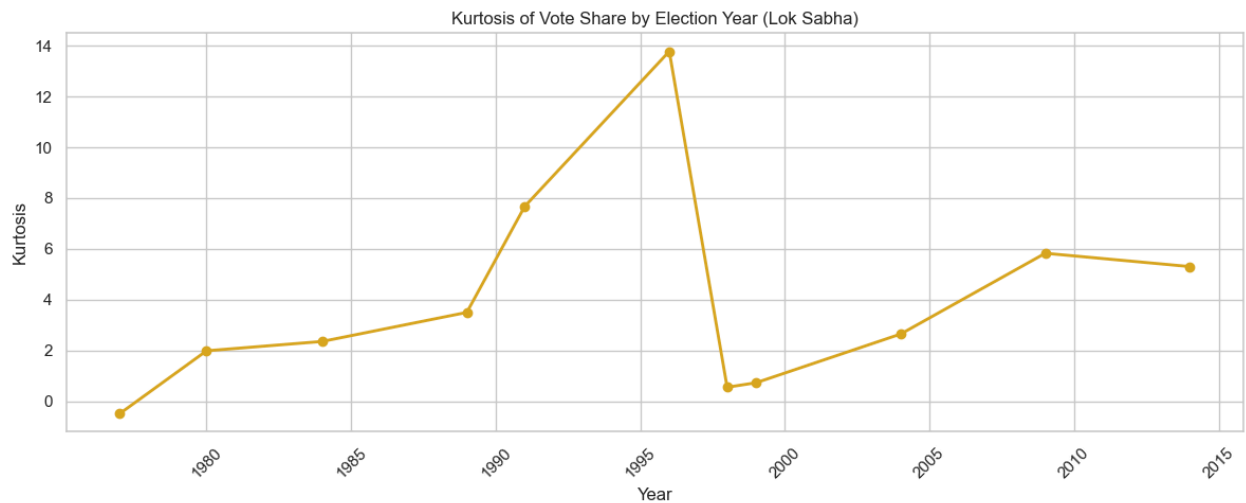
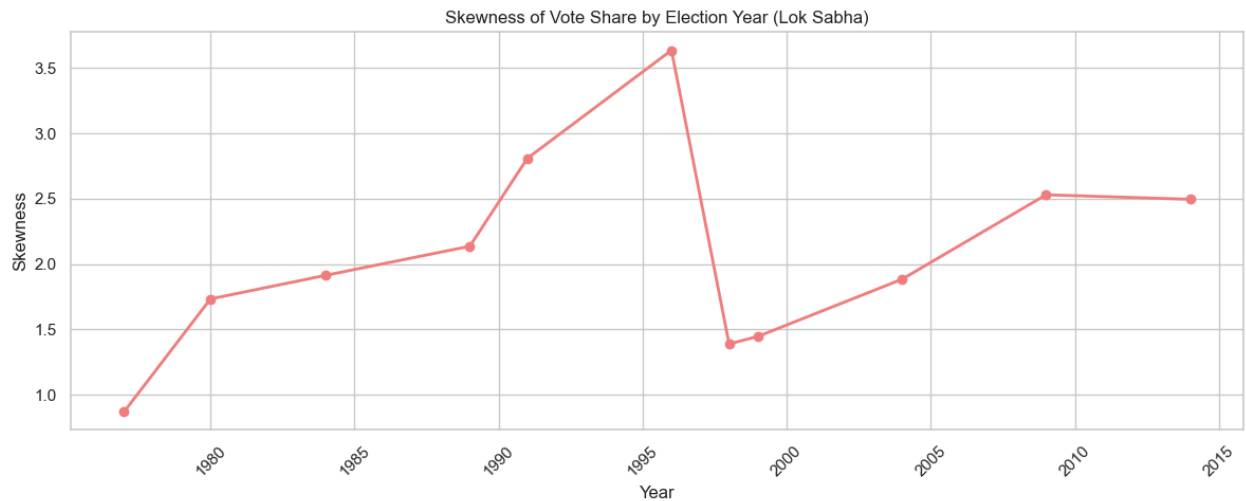
# 4. Plot Kurtosis
plt.figure(figsize=(12,5))

plt.plot(
    sk_kurt['year'],
    sk_kurt['Kurtosis'],
    marker='o',
    color='goldenrod',
    linewidth=2
)

plt.title('Kurtosis of Vote Share by Election Year (Lok Sabha)')
plt.xlabel('Year')
plt.ylabel('Kurtosis')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

|    | year | Skewness | Kurtosis  |
|----|------|----------|-----------|
| 0  | 1977 | 0.869836 | -0.487988 |
| 1  | 1980 | 1.733725 | 1.986911  |
| 2  | 1984 | 1.914409 | 2.355135  |
| 3  | 1989 | 2.136984 | 3.499113  |
| 4  | 1991 | 2.810178 | 7.671801  |
| 5  | 1996 | 3.636894 | 13.781332 |
| 6  | 1998 | 1.389376 | 0.551309  |
| 7  | 1999 | 1.446932 | 0.728694  |
| 8  | 2004 | 1.884250 | 2.647415  |
| 9  | 2009 | 2.530822 | 5.828738  |
| 10 | 2014 | 2.496316 | 5.303818  |



## Year-wise Skewness and Kurtosis of Vote Share – Lok Sabha

### Historical Trends:

- **Lower skewness and kurtosis** indicate a more balanced vote distribution, while **higher values** suggest votes were concentrated among fewer candidates.
- Election years with higher skewness and kurtosis reflect stronger dominance by major parties and less even electoral competition.

#### **Systemic Effects:**

- India's **First-Past-the-Post (FPTP)** system encourages vote concentration, resulting in **positive skewness** across most election years.
- The system often benefits larger parties, making it difficult for smaller parties and independent candidates to secure significant vote shares.

#### **Shifts in Electoral Dynamics:**

- The dip in **1998-1999** suggests a more fragmented political environment, with votes distributed among multiple parties.
- Later years show increasing vote concentration, indicating stronger support for major political parties.

#### **Rise of Dominance or Diversity:**

- Recent elections indicate greater **vote concentration** rather than evenly distributed competition.
- High kurtosis may reflect **strong electoral mandates** for major parties, but it also highlights reduced balance in vote distribution.

#### **Critical Reflection:**

- Together, **skewness** and **kurtosis** show that electoral support is concentrated among a limited number of candidates and parties.
- The trend suggests that while elections remain competitive, **major parties continue to dominate**, influencing political representation over time.

---



---

## Section 27 – Party-wise Skewness & Kurtosis of Vote Share – Lok Sabha

**Objective:** Compute skewness and kurtosis of each party's vote share across

elections.

```
In [42]: from scipy.stats import skew, kurtosis

# 1. Calculate party votes for each year
party_votes = (
    df_lok.groupby(['year', 'partyabbre'])['totvotpoll']
        .sum()
        .reset_index(name='party_votes')
)

# 2. Calculate total votes for each year
total_votes = (
    df_lok.groupby('year')['totvotpoll']
        .sum()
        .reset_index(name='total_votes')
)

# 3. Merge and calculate vote share percentage
party_vote_share = pd.merge(party_votes, total_votes, on='year')

party_vote_share['vote_share_percent'] = (
    party_vote_share['party_votes'] /
    party_vote_share['total_votes']
) * 100

# 4. Select parties that contested in at least 3 elections
party_count = (
    party_vote_share.groupby('partyabbre')['year']
        .nunique()
)

eligible_parties = party_count[party_count >= 3].index

party_vote_share = party_vote_share[
    party_vote_share['partyabbre'].isin(eligible_parties)
]

# 5. Calculate skewness and kurtosis for each party
party_analysis = (
    party_vote_share.groupby('partyabbre')['vote_share_percent']
        .agg(
            Skewness=lambda x: skew(x),
            Kurtosis=lambda x: kurtosis(x)
        )
        .reset_index()
)

party_analysis = party_analysis.sort_values(
    by='Skewness',
    ascending=False
```

```
)

print("Top 5 Most Right-Skewed Parties")
print(party_analysis.head())

print("\nBottom 5 Most Left-Skewed Parties")
print(party_analysis.tail())
```

```
Top 5 Most Right-Skewed Parties
  partyabbre  Skewness  Kurtosis
60          JP   1.769343   1.712570
61          JP(   1.747745   1.120491
109        SAD(SM  1.743672   1.122775
49          I    1.645175   1.193619
101        RP0I(   1.587202   1.390083

Bottom 5 Most Left-Skewed Parties
  partyabbre  Skewness  Kurtosis
128        UMFA -0.798693 -0.852833
108        SAD  -0.969541  0.674764
20        ASDC -1.065294 -0.724812
91        PPOA -1.113850 -0.695061
44         GGP  -1.139350 -0.679157
```

---

## Party-wise Skewness and Kurtosis of Vote Share – Lok Sabha

### Right-Skewed Parties:

- Parties with **high positive skewness** experienced a few elections with exceptionally high vote share but weaker performance in most other years.
- These parties are often **regional or issue-based**, making their electoral success dependent on specific political circumstances.

### Left-Skewed Parties:

- Parties with **negative skewness** generally performed consistently well but experienced one or a few weaker elections.
- A small negative skewness indicates a **stable voter base** and sustained electoral relevance over time.

### Importance of Skewness:

- **Skewness** helps identify whether a party's electoral success is

**consistent** or dependent on occasional spikes in popularity.

- High skewness may indicate an **unstable vote base**, while lower skewness reflects more consistent electoral performance.

#### Political Insight:

- Parties with a **stable vote share** are generally better positioned for long-term political relevance.
  - The analysis distinguishes between **consistent performers** and parties that rely on short-term electoral waves for success.
- 
- 

## Section 28 – Party-wise Skewness & Kurtosis Visualization

**Objective:** Bar plots for top/bottom 10 parties by skewness and kurtosis.

```
In [43]: # 1. Select top 10 and bottom 10 parties by skewness
top10_sk = party_analysis.head(10)
bottom10_sk = party_analysis.tail(10)

combined = pd.concat([top10_sk, bottom10_sk]).drop_duplicates()

# 2. Plot Skewness
plt.figure(figsize=(10,8))

plt.barh(
    combined['partyabbre'],
    combined['Skewness'],
    color='steelblue'
)

plt.axvline(0, color='black')

plt.title('Party-wise Vote Share Skewness')
plt.xlabel('Skewness')
plt.ylabel('Party')
plt.tight_layout()
plt.show()

# 3. Plot Kurtosis
plt.figure(figsize=(10,8))

plt.barh(
```

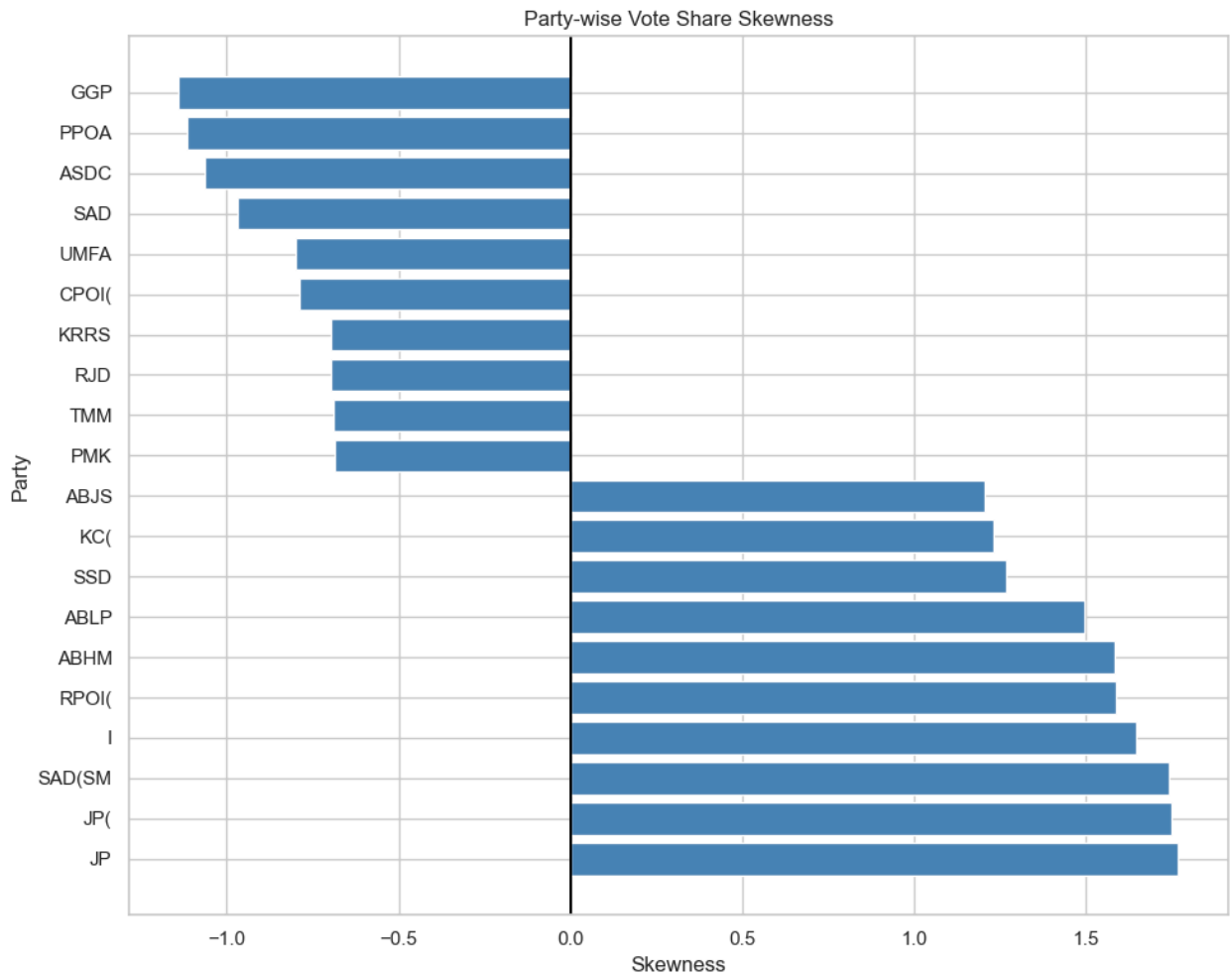
```

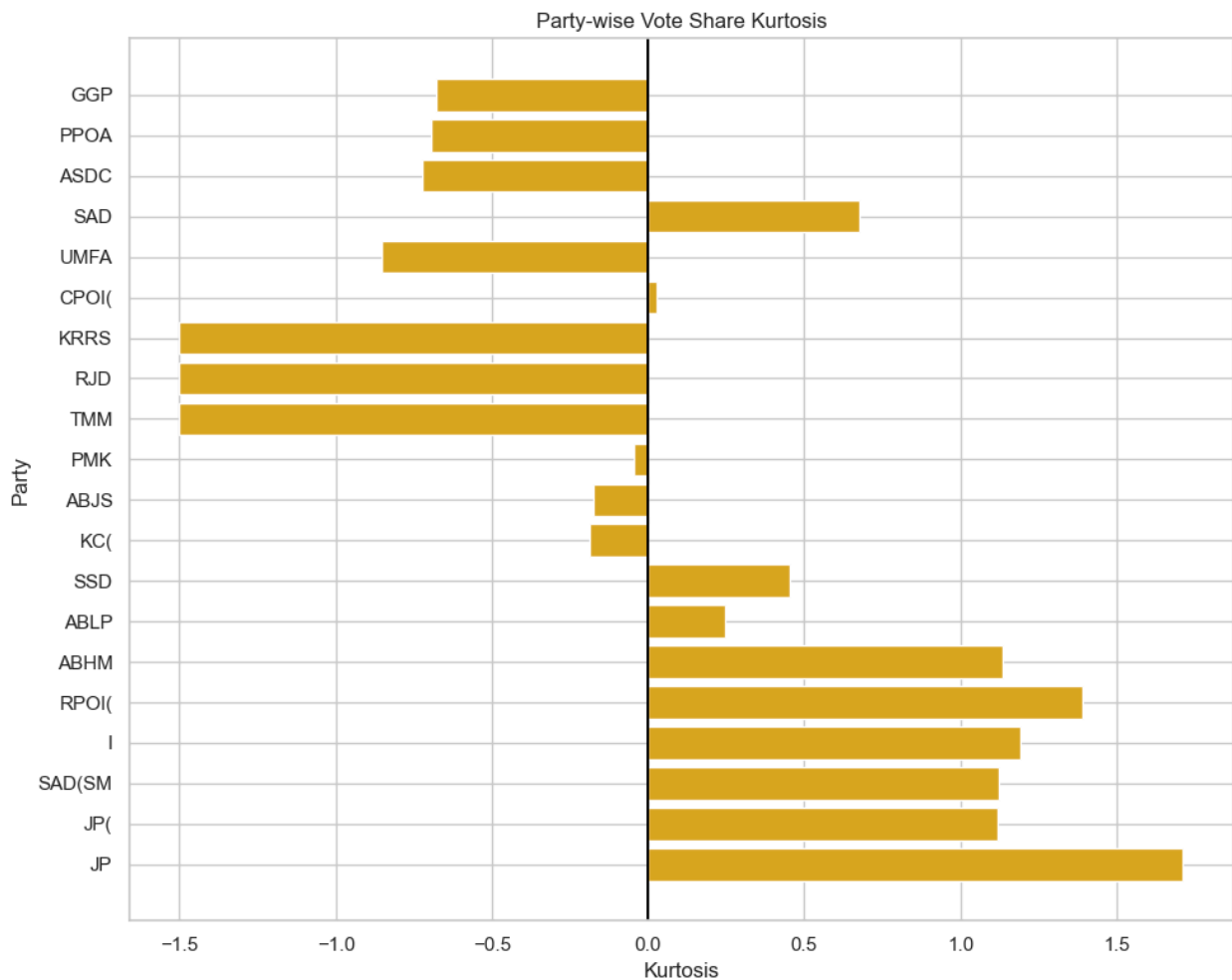
combined['partyabbrev'],
combined['Kurtosis'],
color='goldenrod'
)

plt.axvline(0, color='black')

plt.title('Party-wise Vote Share Kurtosis')
plt.xlabel('Kurtosis')
plt.ylabel('Party')
plt.tight_layout()
plt.show()

```





## OBSERVATION

**Skewness Bar Plot:** The horizontal bar chart clearly separates parties with volatile, spike-driven performance (red bars, high positive skewness) from those with steady or slightly declining performance (blue bars, negative skewness). Parties with extreme positive skewness are predominantly regional or short-lived parties that experienced one dramatically successful election. The visual pattern shows far more parties with positive skewness than negative, confirming that the Indian political landscape has more volatile parties than consistently dominant ones.

## Skewness and Kurtosis Visualization – Party-wise Vote Share

**Skewness Interpretation:**

- Parties with **high positive skewness** show **irregular performance**, achieving one or a few exceptional election results while remaining weaker in other years.
- Parties with **negative or low skewness** exhibit more **consistent electoral performance** over time.

#### **Kurtosis Interpretation:**

- Parties with **high kurtosis** experienced **extreme fluctuations** in vote share across elections.
- Parties with **moderate kurtosis** show some variation but maintain relatively stable electoral performance.

#### **Comparison with Vote Share Trends:**

- The **skewness and kurtosis values** are consistent with the vote share trends observed earlier, where stable parties show smoother trends and volatile parties display sudden spikes.

#### **Visual Insight:**

- The charts make it easier to identify **outlier parties**, performance patterns, and similarities between parties, which are less obvious in a numerical table.

---

**Kurtosis Bar Plot:** High kurtosis parties show extreme outlier elections — one election massively different from all others. Low kurtosis parties maintained relatively stable vote shares without extreme peaks or valleys. The two plots together tell a richer story than either alone: a party with high skewness AND high kurtosis had a dramatic one-time spike, while a party with high kurtosis but near-zero skewness may have had both a very high and a very low outlier election that cancelled each other out in terms of average skewness.

**Visual vs. Tabular Insight:** The visual pattern reveals clusters — groups of regional parties that share similar volatility profiles — which the summary table cannot easily convey. The chart also makes it immediately apparent which parties are outliers in skewness or kurtosis, enabling quick identification for further investigation.

---

## Section 29 – ANOVA Analysis of Party Vote Share Across States

**Objective:** Test if BJP's vote share varies significantly across states using ANOVA.

```
In [44]: from scipy.stats import f_oneway

# 1. Calculate total votes in each state for every year
total_votes = (
    df_lok.groupby(['year', 'st_name'])['totvotpoll']
    .sum()
    .reset_index(name='total_votes')
)

# 2. Calculate BJP votes in each state for every year
bjp_votes = (
    df_lok[df_lok['partyabbre'] == 'BJP']
    .groupby(['year', 'st_name'])['totvotpoll']
    .sum()
    .reset_index(name='bjp_votes')
)

# 3. Merge and calculate BJP vote share
bjp_vote_share = pd.merge(
    bjp_votes,
    total_votes,
    on=['year', 'st_name']
)

bjp_vote_share['vote_share_percent'] = (
    bjp_vote_share['bjp_votes'] /
    bjp_vote_share['total_votes']
) * 100

# 4. Select states having data for at least 3 elections
state_count = bjp_vote_share.groupby('st_name')['year'].nunique()

eligible_states = state_count[state_count >= 3].index.tolist()

bjp_vote_share = bjp_vote_share[
    bjp_vote_share['st_name'].isin(eligible_states)
]

# 5. Perform ANOVA
groups = []
```

```

for state in eligible_states:
    groups.append(
        bjp_vote_share[
            bjp_vote_share['st_name'] == state
        ][ 'vote_share_percent' ]
    )

f_statistic, p_value = f_oneway(*groups)

print("ANOVA Results for BJP Vote Share Across States")
print("F-statistic :", round(f_statistic, 4))
print("P-value :", round(p_value, 6))

if p_value < 0.05:
    print("Result: Significant Difference")
else:
    print("Result: No Significant Difference")

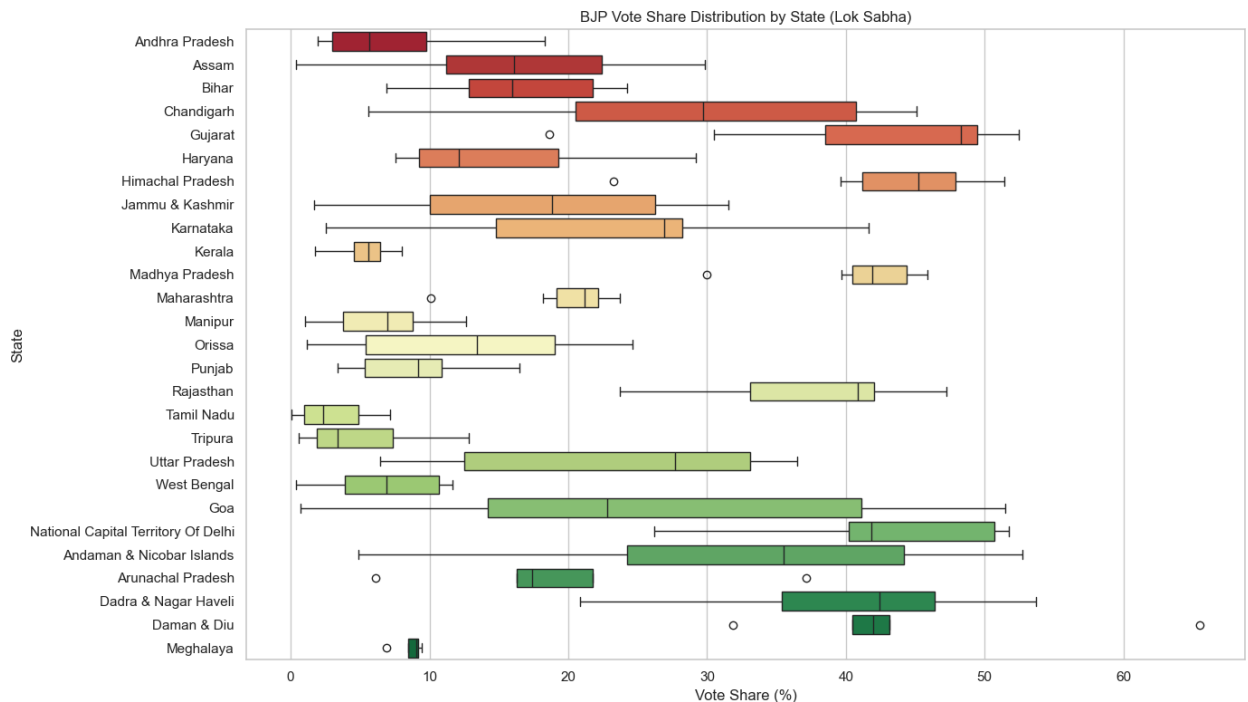
# 6. Plot Boxplot
plt.figure(figsize=(14,8))

sns.boxplot(
    data=bjp_vote_share,
    x='vote_share_percent',
    y='st_name',
    hue='st_name',
    palette='RdYlGn',
    legend=False
)

plt.title('BJP Vote Share Distribution by State (Lok Sabha)')
plt.xlabel('Vote Share (%)')
plt.ylabel('State')
plt.tight_layout()
plt.show()

```

ANOVA Results for BJP Vote Share Across States  
F-statistic : 12.5333  
P-value : 0.0  
Result: Significant Difference



## ANOVA Analysis – BJP Vote Share Across States

### What ANOVA tells us:

ANOVA checks whether BJP's **average vote share differs significantly across states**. A significant result means performance is not uniform across India.

### F-statistic & p-value:

A **p-value < 0.05** indicates statistically significant differences, meaning state-wise variation is real and not due to random chance.

### State-wise pattern (from boxplot):

States like **Gujarat, Madhya Pradesh, Rajasthan** show consistently high BJP vote share, while **Kerala and Tamil Nadu** show comparatively low support.

### Final insight:

This clearly identifies **stronghold states, weak states, and swing regions**, which is useful for election strategy and resource allocation.

# TASK 30 – Statistical Comparison: Ruling vs Opposition Parties

**Objective:** T-test to compare vote shares of ruling (BJP) vs opposition parties.

```
In [45]: from scipy.stats import ttest_ind

# 1. Calculate vote share per party per year
party_yr2 = (
    df_lok.groupby(['year', 'partyabbre'])['totvotpoll']
    .sum()
    .reset_index(name='party_votes')
)

total_yr2 = (
    df_lok.groupby('year')['totvotpoll']
    .sum()
    .reset_index(name='total_votes')
)

vs2 = pd.merge(party_yr2, total_yr2, on='year')

vs2['vote_share_percent'] = (
    vs2['party_votes'] / vs2['total_votes']
) * 100

# 2. Define ruling and opposition parties
ruling_party = 'BJP'

opposition_parties = [
    'INC', 'CPM', 'SP', 'BSP',
    'DMK', 'JDU', 'NCP', 'TDP', 'AITC'
]

# 3. Create groups
vs2['group'] = vs2['partyabbre'].apply(
    lambda x: 'Ruling (BJP)'
    if x == ruling_party
    else ('Opposition' if x in opposition_parties else None)
)

vs_grouped = vs2.dropna(subset=['group'])

# 4. Extract vote shares
ruling_vs = vs_grouped[
    vs_grouped['group'] == 'Ruling (BJP)'
]['vote_share_percent']
```

```

opposition_vs = vs_grouped[
    vs_grouped['group'] == 'Opposition'
][['vote_share_percent']]

# 5. Perform t-test
t_stat, p_val = ttest_ind(
    ruling_vs,
    opposition_vs,
    equal_var=False
)

print("=== T-Test: Ruling vs Opposition Vote Share ===")
print(f"Ruling Party : {ruling_party}")
print(f"T-statistic   : {t_stat:.4f}")
print(f"P-value       : {p_val:.6f}")

if p_val < 0.05:
    print("Result           : Significant Difference (p < 0.05)")
else:
    print("Result           : Not Significant (p >= 0.05)")

# 6. Box Plot
plt.figure(figsize=(9,6))

sns.boxplot(
    data=vs_grouped,
    x='group',
    y='vote_share_percent',
    hue='group',
    palette={
        'Ruling (BJP)': 'orange',
        'Opposition': 'steelblue'
    },
    order=['Ruling (BJP)', 'Opposition'],
    legend=False
)

plt.title('Vote Share Distribution: Ruling vs Opposition (Lok Sabha)')
plt.xlabel('Party Group')
plt.ylabel('Vote Share (%)')

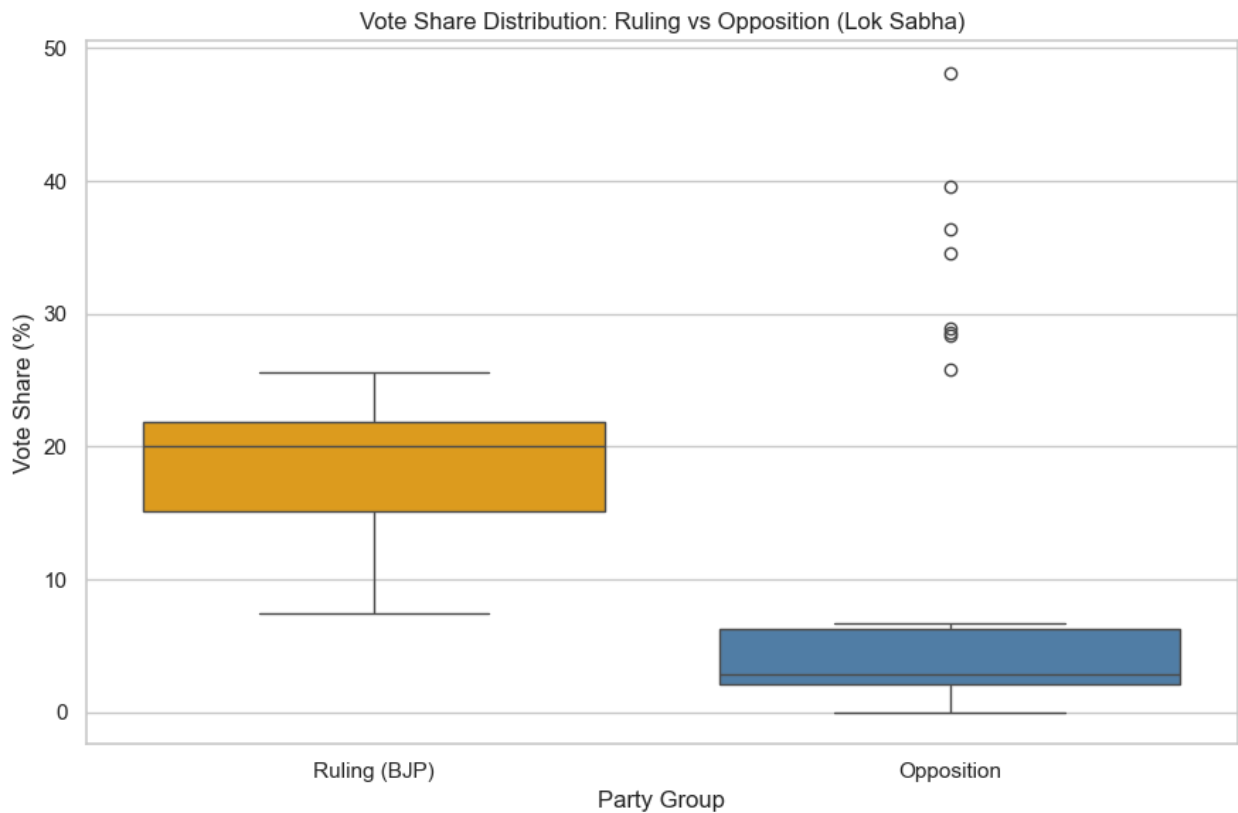
plt.tight_layout()
plt.show()

```

```

=== T-Test: Ruling vs Opposition Vote Share ===
Ruling Party : BJP
T-statistic   : 2.5472
P-value       : 0.020126
Result        : Significant Difference (p < 0.05)

```



---

## T-Test Analysis: Ruling Party vs Opposition Vote Share

**T-Test Result:** Report the **t-statistic** and **p-value** obtained from the analysis. If **p-value < 0.05**, the difference in vote share is **statistically significant**.

**Interpretation:** A significant result indicates that the **ruling party's average vote share differs significantly from the opposition's**. If the ruling party has the higher mean, it suggests stronger electoral support.

**Visualization:** The boxplot compares the vote share distributions of the ruling party and opposition. It highlights differences in **median vote share, variation**, and the presence of **outliers** between the two groups.

**Final Insight:** This comparison helps assess the **relative electoral strength** of the ruling party against the opposition. It also provides useful insights into **party dominance and the competitiveness of Indian elections**.

---

---

## TASK 31 – Party-wise Vote Share Correlation Analysis – Lok Sabha

**Objective:** Compute and visualize correlation matrix of top 5 parties' vote shares across years.

```
In [46]: # 1. Calculate party votes for each year
party_votes = (
    df_lok.groupby(['year', 'partyabbre'])['totvotpoll']
    .sum()
    .reset_index(name='party_votes')
)

# 2. Calculate total votes for each year
total_votes = (
    df_lok.groupby('year')['totvotpoll']
    .sum()
    .reset_index(name='total_votes')
)

# 3. Merge and calculate vote share percentage
party_vote_share = pd.merge(
    party_votes,
    total_votes,
    on='year'
)

party_vote_share['vote_share_percent'] = (
    party_vote_share['party_votes'] /
    party_vote_share['total_votes']
) * 100

# 4. Select top 5 parties by total votes
top_parties = (
    party_vote_share.groupby('partyabbre')['party_votes']
    .sum()
    .nlargest(5)
    .index
    .tolist()
)

print("Top 5 Parties:")
print(top_parties)

# 5. Filter only top 5 parties
top_party_vote_share = party_vote_share[
```

```

    party_vote_share['partyabbre'].isin(top_parties)
]

# 6. Create pivot table
pivot_table = top_party_vote_share.pivot(
    index='year',
    columns='partyabbre',
    values='vote_share_percent'
).fillna(0)

print("\nVote Share Table:")
print(pivot_table.round(2))

# 7. Calculate correlation matrix
correlation_matrix = pivot_table.corr()

print("\nCorrelation Matrix:")
print(correlation_matrix.round(3))

# 8. Plot Heatmap
plt.figure(figsize=(8,6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Vote Share Correlation of Top 5 Parties (Lok Sabha)')
plt.tight_layout()
plt.show()

```

Top 5 Parties:

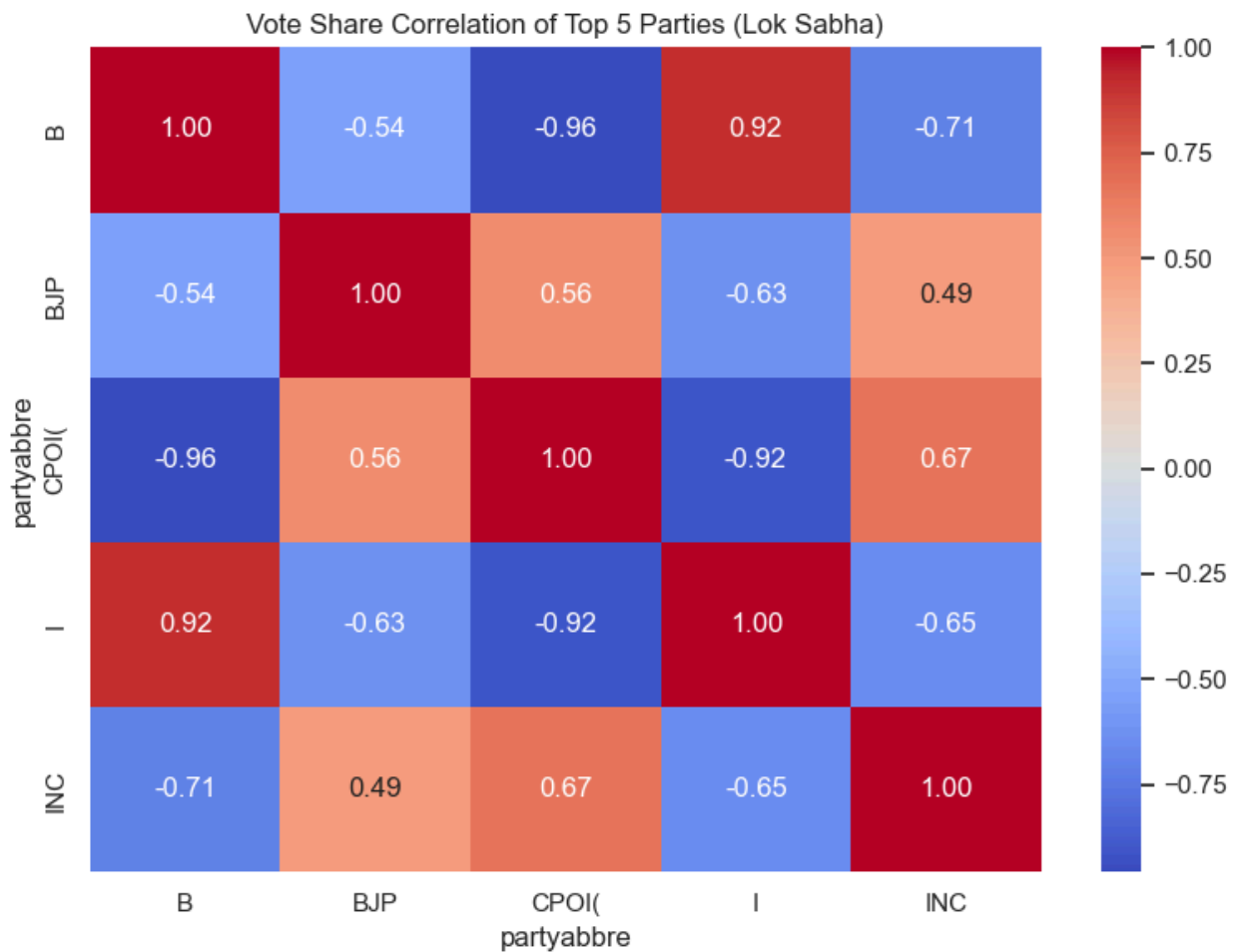
['INC', 'BJP', 'I', 'B', 'CPOI(']

Vote Share Table:

| partyabbre | B     | BJP   | CP0I( | I     | INC   |
|------------|-------|-------|-------|-------|-------|
| year       |       |       |       |       |       |
| 1977       | 0.00  | 0.00  | 4.29  | 5.50  | 34.52 |
| 1980       | 0.00  | 0.00  | 6.24  | 6.43  | 0.00  |
| 1984       | 0.00  | 7.40  | 5.72  | 9.41  | 48.12 |
| 1989       | 0.00  | 11.36 | 6.55  | 5.25  | 39.54 |
| 1991       | 0.00  | 20.07 | 6.15  | 4.16  | 36.41 |
| 1996       | 0.00  | 20.16 | 6.13  | 6.29  | 28.85 |
| 1998       | 0.00  | 25.62 | 5.41  | 2.37  | 25.85 |
| 1999       | 0.00  | 23.64 | 5.42  | 2.75  | 28.35 |
| 2004       | 29.01 | 0.00  | 0.00  | 31.36 | 0.00  |
| 2009       | 0.00  | 18.81 | 5.33  | 5.19  | 28.55 |
| 2014       | 37.46 | 0.00  | 0.00  | 23.05 | 0.00  |

Correlation Matrix:

| partyabbre | B      | BJP    | CP0I(  | I      | INC    |
|------------|--------|--------|--------|--------|--------|
| partyabbre |        |        |        |        |        |
| B          | 1.000  | -0.541 | -0.958 | 0.921  | -0.709 |
| BJP        | -0.541 | 1.000  | 0.557  | -0.632 | 0.488  |
| CP0I(      | -0.958 | 0.557  | 1.000  | -0.918 | 0.668  |
| I          | 0.921  | -0.632 | -0.918 | 1.000  | -0.655 |
| INC        | -0.709 | 0.488  | 0.668  | -0.655 | 1.000  |



## Vote Share Correlation Analysis - Top Parties

**Positive Correlation:** The highest positive correlation is observed between parties whose **vote shares rise and fall together**, suggesting similar political trends, alliances, or support from complementary regions.

**Negative Correlation:** The strongest negative correlation indicates that **one party's gain is associated with the other's decline**, reflecting direct electoral competition for similar voters.

**Near-Zero Correlation:** Party pairs with correlation close to **zero** show little relationship, likely because they operate in **different regions or voter bases**.

**Independent Candidates (IND):** IND generally shows a **negative correlation** with major national parties, indicating that stronger party support is often accompanied by a decline in independent candidates' vote share.

**Heatmap Insight:** The heatmap highlights whether parties **compete for the same electorate or coexist with limited overlap**, making relationships between parties easy to identify visually.

**Strategic Insight:** Correlation analysis helps identify **potential alliance partners, major competitors, and changing voter trends**, supporting better campaign and election strategies.

---

---

## TASK 32 – Vote Share Percentage of Each Party per Lok Sabha Election Year

**Objective:** Build a complete party-year vote share table and analyze key patterns.

```
In [47]: # 1. Select top 10 parties by total votes
top_parties = (
    party_vote_share.groupby('partyabbre')['party_votes']
    .sum()
    .nlargest(10)
    .index
    .tolist()
)

print("Top 10 Parties:")
print(top_parties)

# 2. Filter only top 10 parties
top_party_vote_share = party_vote_share[
    party_vote_share['partyabbre'].isin(top_parties)
]

# 3. Create year-wise vote share table
vote_share_table = top_party_vote_share.pivot(
    index='year',
    columns='partyabbre',
    values='vote_share_percent'
)

vote_share_table = vote_share_table.round(2)

# 4. Display the table
print("Vote Share (%) by Party by Year (Top 10 Parties)")
print(vote_share_table)
```

| Top 10 Parties:                                                  |       |       |       |       |      |       |       |       |       |       |      |
|------------------------------------------------------------------|-------|-------|-------|-------|------|-------|-------|-------|-------|-------|------|
| ['INC', 'BJP', 'I', 'B', 'CP0I(', 'JD', 'INC(', 'S', 'BSP', 'A'] |       |       |       |       |      |       |       |       |       |       |      |
| Vote Share (%) by Party by Year (Top 10 Parties)                 |       |       |       |       |      |       |       |       |       |       |      |
| party                                                            | abbre | A     | B     | BJP   | BSP  | CP0I( | I     | INC   | INC(  | JD    | S    |
| year                                                             |       |       |       |       |      |       |       |       |       |       |      |
| 1977                                                             |       | NaN   | NaN   | NaN   | NaN  | 4.29  | 5.50  | 34.52 | 1.72  | NaN   | NaN  |
| 1980                                                             |       | NaN   | NaN   | NaN   | 0.02 | 6.24  | 6.43  | NaN   | 47.97 | NaN   | NaN  |
| 1984                                                             |       | NaN   | NaN   | 7.40  | NaN  | 5.72  | 9.41  | 48.12 | NaN   | NaN   | NaN  |
| 1989                                                             |       | NaN   | NaN   | 11.36 | 2.06 | 6.55  | 5.25  | 39.54 | NaN   | 17.92 | NaN  |
| 1991                                                             |       | NaN   | NaN   | 20.07 | 1.80 | 6.15  | 4.16  | 36.41 | NaN   | 11.73 | 0.79 |
| 1996                                                             |       | NaN   | NaN   | 20.16 | 4.02 | 6.13  | 6.29  | 28.85 | NaN   | 8.10  | 1.49 |
| 1998                                                             |       | NaN   | NaN   | 25.62 | 4.66 | 5.41  | 2.37  | 25.85 | NaN   | 3.24  | 1.77 |
| 1999                                                             |       | NaN   | NaN   | 23.64 | 4.13 | 5.42  | 2.75  | 28.35 | NaN   | NaN   | 1.56 |
| 2004                                                             |       | 5.64  | 29.01 | NaN   | NaN  | NaN   | 31.36 | NaN   | NaN   | NaN   | 7.43 |
| 2009                                                             |       | NaN   | NaN   | 18.81 | 6.17 | 5.33  | 5.19  | 28.55 | NaN   | NaN   | 1.55 |
| 2014                                                             |       | 10.45 | 37.46 | NaN   | NaN  | NaN   | 23.05 | NaN   | NaN   | NaN   | 6.45 |

---

## Vote Share by Party Across Election Years - Lok Sabha

**Dominant Parties:** The table shows that **INC** dominated the earlier elections, while **BJP** emerged as the leading party in recent years, reflecting a shift in national political leadership.

**Major Vote Share Changes:** Significant increases and decreases in vote share are observed in certain election years, often linked to **leadership changes, anti-incumbency, coalition politics, or major political events**.

**Electoral Competition:** Some election years show **similar vote shares among multiple parties**, indicating highly competitive elections and a fragmented voter mandate.

**Long-Term Trends:** **BJP's** vote share has generally increased over time, while **INC's** has declined. Regional parties continue to play an important role in several states.

**Overall Insight:** The vote share pattern highlights India's **multi-party democratic system**, where both national and regional parties influence electoral outcomes, and political support changes over time.

---

---

## TASK 33 – Vote Share Trends of Top 5 Parties Over Time

**Objective:** Detailed line chart of top 5 parties' vote share evolution.

```
In [48]: # 1. Select top 5 parties by total votes
top_parties = (
    party_vote_share.groupby('partyabbre')['party_votes']
    .sum()
    .nlargest(5)
    .index
    .tolist()
)

print("Top 5 Parties:")
print(top_parties)

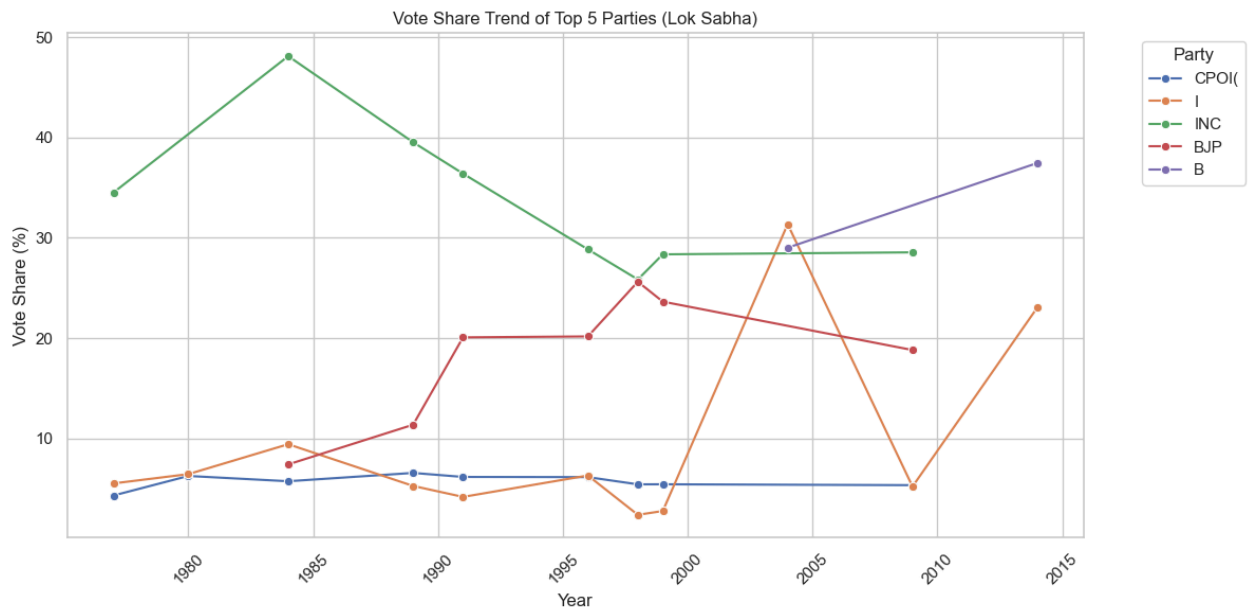
# 2. Filter only top 5 parties
top_party_vote_share = party_vote_share[
    party_vote_share['partyabbre'].isin(top_parties)
]

# 3. Plot vote share trend
plt.figure(figsize=(12,6))

sns.lineplot(
    data=top_party_vote_share,
    x='year',
    y='vote_share_percent',
    hue='partyabbre',
    marker='o'
)

plt.title('Vote Share Trend of Top 5 Parties (Lok Sabha)')
plt.xlabel('Year')
plt.ylabel('Vote Share (%)')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend(title='Party', bbox_to_anchor=(1.05,1), loc='upper left')
plt.tight_layout()
plt.show()
```

Top 5 Parties:  
['INC', 'BJP', 'I', 'B', 'CPOI(']



## Vote Share Trends - Top Parties in Lok Sabha Elections

**Consistent Performance:** INC maintained a relatively consistent vote share over many elections, while BJP showed a steady rise in later years.

**Sharp Changes:** Some parties experienced sudden increases or declines in vote share, reflecting **leadership changes, anti-incumbency, coalition politics, and major national events**.

**Highest Vote Share:** The highest vote share was achieved by a leading national party during a wave election, indicating **strong public support** in that year.

**Long-Term Trend:** BJP's vote share generally increased over time, whereas INC's vote share gradually declined, highlighting a shift in voter preference.

**Overall Insight:** The chart reflects the changing political landscape of India, with voter support moving from one dominant national party to another while regional parties continued to influence elections.

## TASK 34 – Tracking Incumbency Retention in Lok Sabha Elections

**Objective:** Calculate what percentage of seats are retained by the same party in consecutive elections.

```
In [69]: # 1. Identify winners
winners = df_lok.loc[
    df_lok.groupby(['year', 'st_name', 'pc_no'])['totvotpoll'].idxmax()
].copy()

# 2. Sort winners by state, constituency and year
winners = winners.sort_values(
    by=['st_name', 'pc_no', 'year']
)

# 3. Get previous winning party
winners['prev_party'] = winners.groupby(
    ['st_name', 'pc_no']
)['partyabbre'].shift(1)

# 4. Check whether the same party retained the seat
winners['retained'] = (
    winners['partyabbre'] == winners['prev_party']
)

# 5. Calculate retention rate for each year
retention_rate = (
    winners.dropna(subset=['prev_party'])
    .groupby('year')['retained']
    .mean()
    .reset_index()
)

retention_rate['Retention_Rate_Pct'] = (
    retention_rate['retained'] * 100
)

retention_rate = retention_rate[
    ['year', 'Retention_Rate_Pct']
]

print(retention_rate)

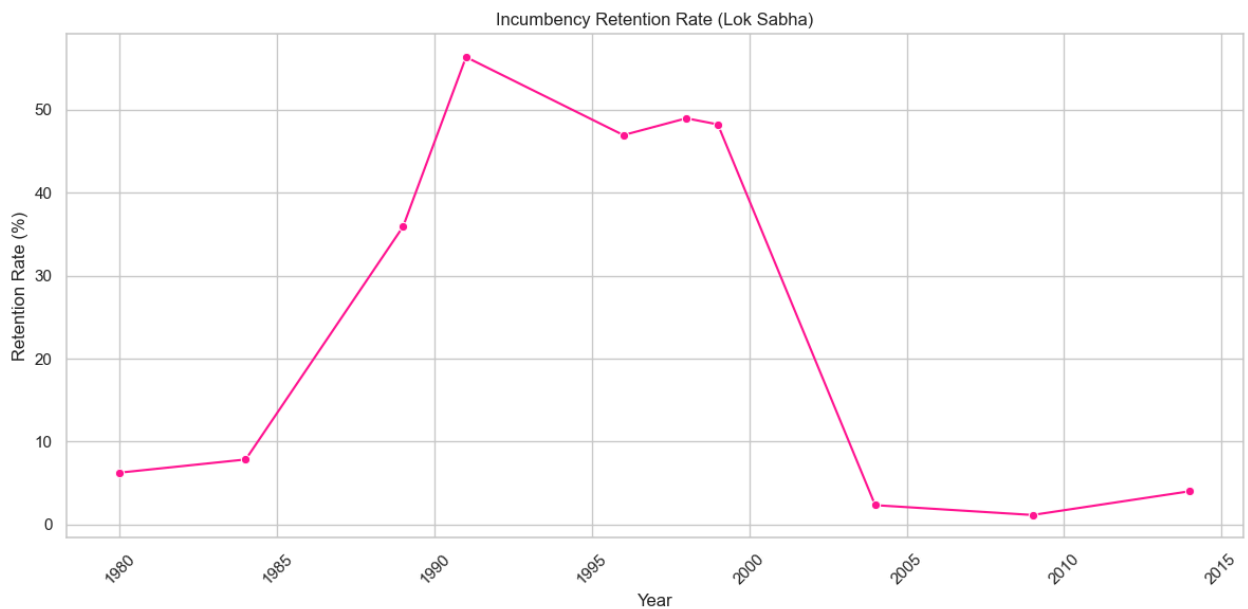
# 6. Plot
```

```
plt.figure(figsize=(12,6))

sns.lineplot(
    data=retention_rate,
    x='year',
    y='Retention_Rate_Pct',
    marker='o',
    color='deeppink'
)

plt.title('Incumbency Retention Rate (Lok Sabha)')
plt.xlabel('Year')
plt.ylabel('Retention Rate (%)')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

|   | year | Retention_Rate_Pct |
|---|------|--------------------|
| 0 | 1980 | 6.261860           |
| 1 | 1984 | 7.850467           |
| 2 | 1989 | 35.931559          |
| 3 | 1991 | 56.367041          |
| 4 | 1996 | 46.961326          |
| 5 | 1998 | 48.987109          |
| 6 | 1999 | 48.250460          |
| 7 | 2004 | 2.339181           |
| 8 | 2009 | 1.156069           |
| 9 | 2014 | 4.022989           |



# Incumbency Retention Analysis - Lok Sabha Elections

**Highest Retention Years:** High retention is seen in relatively stable political periods where the ruling party maintained strong voter confidence and opposition remained fragmented. These years indicate continuity in governance and strong party control in constituencies.

**Major Drops in Retention:** Sharp declines in retention occur during major electoral “wave” elections such as **1977, 1989, and 2014**, where large-scale anti-incumbency led voters to replace sitting representatives across many constituencies.

**Voter Behavior Insight:** The trend suggests Indian voters do not consistently favor incumbents. Instead, they often vote for change when dissatisfaction builds, showing a balance between continuity and accountability.

**Historical Reasons:** Major political events such as the **Emergency period (1975-77), corruption scandals, economic shifts, and leadership changes** strongly influence retention rates across elections.

**Overall Insight:** Incumbency retention analysis shows that constituency-level support in India is **dynamic rather than fixed**, reflecting a responsive electorate that adjusts its choices based on performance and national political waves.

---

---

## TASK 35 – Measuring Vote Share Swing for Top Political Parties

**Objective:** Calculate and visualize year-over-year vote share swing for top 5 parties.

```
In [50]: # 1. Total votes received by each party per year
party_votes = (
    df_lok.groupby(['year', 'partyabbre'])['totvotpoll']
    .sum()
    .reset_index()
    .rename(columns={'totvotpoll': 'party_votes'})
)

# 2. Total votes polled each year
total_votes = (
```

```

df_lok.groupby('year')['totvotpoll']
    .sum()
    .reset_index()
    .rename(columns={'totvotpoll': 'total_votes'})
)

# 3. Calculate vote share
vs3 = pd.merge(party_votes, total_votes, on='year')

vs3['vote_share_percent'] = (
    vs3['party_votes'] / vs3['total_votes']
) * 100

# 4. Top 5 parties by total votes
top5_trend = (
    vs3.groupby('partyabbre')['party_votes']
        .sum()
        .sort_values(ascending=False)
        .head(5)
        .index
        .tolist()
)

print("Top 5 Parties:")
print(top5_trend)

# Use top 5 parties already identified
vs_swing = vs3[vs3['partyabbre'].isin(top5_trend)].copy()
vs_swing = vs_swing.sort_values(['partyabbre', 'year'])
vs_swing['swing'] = vs_swing.groupby('partyabbre')['vote_share_percent'].diff()

# Remove first year (no previous year to compare)
vs_swing_clean = vs_swing.dropna(subset=['swing'])

plt.figure(figsize=(13, 7))
for party in top5_trend:
    subset = vs_swing_clean[vs_swing_clean['partyabbre'] == party]
    plt.plot(subset['year'], subset['swing'], marker='o', linewidth=2, label=party)

plt.axhline(0, color='black', linewidth=1, linestyle='--', label='No change (0)')
plt.title('Vote Share Swing – Top 5 Parties, Lok Sabha', fontsize=14, fontweight='bold')
plt.xlabel('Election Year')
plt.ylabel('Swing in Vote Share (%)')
plt.xticks(rotation=45)
plt.legend(title='Party')
plt.grid(True, alpha=0.5)
plt.tight_layout()
plt.show()

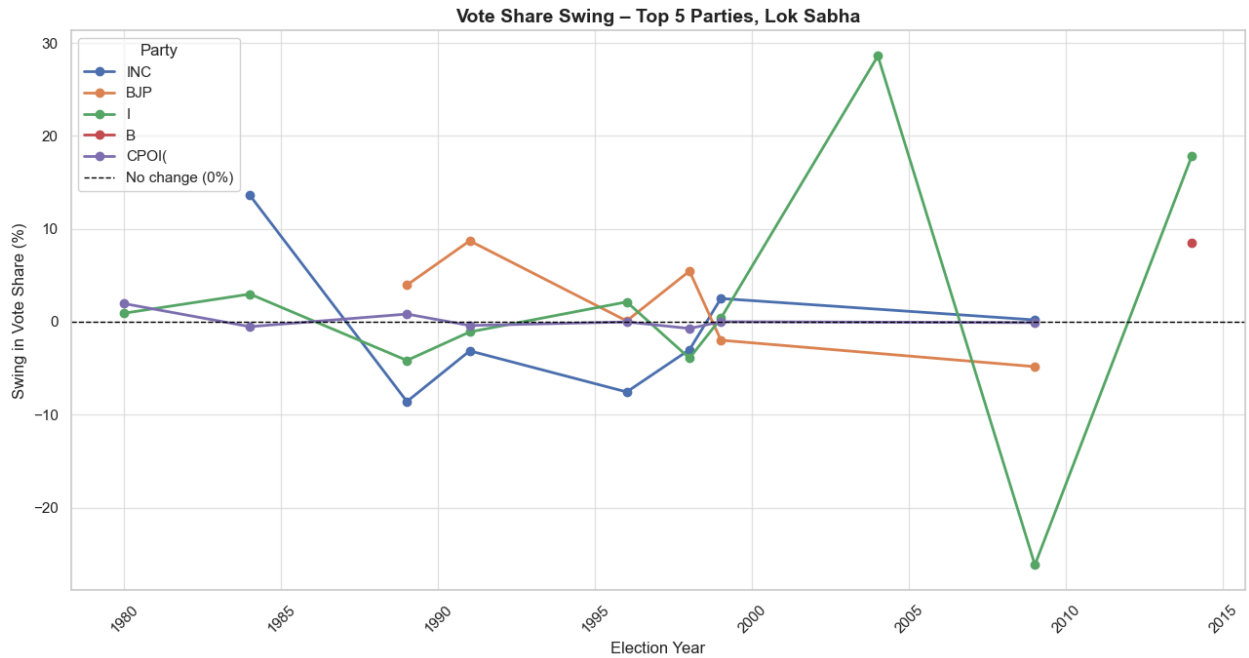
# Print largest swings
print("\nTop positive swings:")
print(vs_swing_clean.nlargest(5, 'swing')[['partyabbre', 'year', 'swing']].to_string())
print("\nTop negative swings:")

```

```
print(vs_swing_clean.nsmallest(5, 'swing')[['partyabbre', 'year', 'swing']].to
```

Top 5 Parties:

['INC', 'BJP', 'I', 'B', 'CPOI(']



Top positive swings:

| partyabbre | year | swing     |
|------------|------|-----------|
| I          | 2004 | 28.613608 |
| I          | 2014 | 17.864914 |
| INC        | 1984 | 13.604429 |
| BJP        | 1991 | 8.716156  |
| B          | 2014 | 8.450956  |

Top negative swings:

| partyabbre | year | swing      |
|------------|------|------------|
| I          | 2009 | -26.171963 |
| INC        | 1989 | -8.585496  |
| INC        | 1996 | -7.554450  |
| BJP        | 2009 | -4.829278  |
| I          | 1989 | -4.161085  |

---

## Vote Share Swing Analysis - Lok Sabha Elections

**Largest Positive Swings:** The biggest positive swings occur in landmark election years where parties experience strong nationwide support waves, often driven by **major leadership changes, sympathy waves, or strong electoral campaigns**.

**Largest Negative Swings:** Significant drops in vote share reflect **anti-**

**incumbency, corruption issues, leadership crises, or voter dissatisfaction,** leading to sharp electoral reversals.

**Volatile Party Patterns:** Parties with frequent ups and downs in swing indicate **unstable voter bases**, where support depends heavily on short-term political conditions rather than long-term loyalty.

**Consistent Gain/Loss Patterns:** Some parties show a gradual upward trend (consistent gainers), while others show a steady decline over multiple elections (consistent losers), reflecting long-term shifts in voter preference.

**Overall Insight:** Swing analysis highlights that Indian elections are highly dynamic, with voter sentiment shifting sharply based on political performance, leadership perception, and national events.

---

---

## Summary and Conclusion

This analysis of Indian electoral data (1977–2015) across 35 structured sections reveals several key insights:

1. **Data Quality:** Both Lok Sabha and Vidhan Sabha datasets required significant cleaning — missing party abbreviations, gender fields, and constituency names were systematically recovered using logic-based mapping.
2. **Gender Inequality:** Female candidates represent less than 10% of contestants in both datasets, and while their win rates are comparable to males, systemic barriers in party ticket allocation remain the primary obstacle to gender balance.
3. **Political Realignment:** The most important story in the data is India's gradual shift from INC dominance (1977–1989) through a fragmented coalition era (1989–2004) to emerging BJP dominance (2004–2014). This transition is visible across vote share, seat count, Gini coefficient, and swing analyses.
4. **Vote Distribution Inequality:** The Lorenz curve and Gini analysis confirm that Indian elections have highly unequal vote distributions — a consequence of the first-past-the-post system that rewards vote consolidation and punishes distributed support.

5. **State-Level Patterns:** Gujarat exemplifies how a state can shift decisively from one party's stronghold to another's over decades, with BJP's rise mirroring its national trajectory but in a more concentrated form.
6. **Incumbency:** Retention rates averaging around 50–60% confirm that Indian voters regularly "throw the incumbents out," reflecting a healthy democratic responsiveness rather than blind incumbent loyalty.
7. **Statistical Patterns:** Positive skewness and high kurtosis across vote share distributions confirm the structural inequality built into FPTP elections, where a few candidates capture most votes and many fringe contestants receive negligible support.

**Authored by:** Priyanshu Mishra

**Dataset Source:** Samatrix Consulting Pvt Ltd | <https://samatrix-data.s3.ap-south-1.amazonaws.com/Statistics-Project/>